

UNIVERSITÀ DI PISA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Laurea Triennale in Ingegneria Informatica

Progettazione e realizzazione di un monitor di accessi a file basato su BPF-LSM nel kernel Linux

Relatori:

Prof. Giuseppe Lettieri

Dott. Fabio Piras

Candidato:

Luca Maffioli

ANNO ACCADEMICO 2025/2026

Sommario

La tesi affronta il monitoraggio degli accessi a file in un sistema Linux, attraverso lo sviluppo di un programma eBPF e l'utilizzo di hook del framework Linux Security Modules (LSM), con l'obiettivo di osservare e segnalare operazioni che coinvolgono file sensibili.

Un aspetto centrale riguarda il riconoscimento affidabile delle risorse da sorvegliare. Un file può essere indicato attraverso il suo path, cioè il riferimento usato nella configurazione, oppure attraverso la coppia device e inode, che individua l'oggetto corrispondente nel filesystem. Nessuna delle due informazioni è però sufficiente da sola. Il path non riconosce gli accessi eseguiti con un altro nome, ad esempio tramite un hard link, e nei container, che hanno un mount namespace diverso da quello dell'host, può indicare un altro oggetto. L'inode invece può cambiare quando il file viene sostituito, come accade al salvataggio con molti editor, e può produrre falsi positivi se il suo numero viene riassegnato.

Per affrontare questi problemi il prototipo è stato sviluppato per versioni successive, ognuna verificata con una serie di casi di test e motivata dai limiti emersi nella precedente. La versione finale usa un meccanismo ibrido, in cui il path è utilizzato come riferimento logico della risorsa da proteggere e la coppia device e inode per riconoscere l'oggetto che le corrisponde. Il sistema mantiene dinamicamente l'associazione tra i due intercettando, oltre all'apertura dei file, anche le operazioni di rinomina, cancellazione e creazione di link. Viene inoltre tenuto conto del mount namespace da cui viene effettuato l'accesso, in modo che la risorsa venga riconosciuta correttamente anche in presenza di container.

Il prototipo è stato validato con una serie di test che comprendono la sostituzione e la rinomina dei file, gli hard link e i link simbolici, il riutilizzo degli inode e gli accessi da container. I risultati mostrano che tenendo insieme queste informazioni è possibile mantenere il riferimento ai file sorvegliati anche quando il filesystem viene modificato o vi si accede da viste diverse.

Indice

Sommario **3**

1 Introduzione **7**
1.1 Contesto . . . 7
1.2 Strumenti esistenti . . . 7
1.3 Obiettivo del lavoro . . . 8
1.4 Metodo . . . 8

2 eBPF e libbpf **11**
2.1 eBPF . . . 11
2.2 Verifier, funzioni helper e mappe . . . 11
2.3 Linux Security Modules . . . 12
2.4 BTF e CO-RE . . . 13
2.5 libbpf . . . 13
2.6 Struttura del prototipo . . . 13
2.7 Ambiente di sviluppo . . . 14

3 Identificare un file **15**
3.1 Il filesystem visto dal kernel . . . 15
3.2 Percorso e oggetto . . . 16
3.3 Semantiche possibili per una policy . . . 17

4 Strategie di identificazione a confronto **19**
4.1 Architettura iniziale . . . 19
4.2 Il problema dell'editor . . . 20
4.3 Aggiunta del criterio per path . . . 21
4.3.1 Risoluzione del path . . . 21
4.3.2 Vincoli imposti dal verifier e memoria di appoggio . . . 21
4.4 Test per il confronto . . . 22
4.5 Risultati . . . 22
4.6 Considerazioni . . . 24

5 Strategia ibrida **25**
5.1 Il path come riferimento principale . . . 25

5.2 Riallineamento all'apertura . . . 26
5.3 Seguire le modifiche del filesystem . . . 28
5.3.1 Comporre il path senza `struct file` . . . 29
5.3.2 `path_unlink` . . . 29
5.3.3 `path_rename` . . . 30
5.3.4 `path_link` . . . 30
5.4 La semantica scelta per gli hard link . . . 31
5.5 Test . . . 31
5.6 Risultati . . . 32
5.7 Considerazioni . . . 35

6 Container **37**
6.1 Namespace e container . . . 37
6.2 Primi test . . . 38
6.3 Revisione del prototipo . . . 39
6.4 Implementazione . . . 41
6.5 Risultati . . . 42
6.6 Considerazioni . . . 45

7 Conclusioni **47**
7.1 Limiti noti . . . 47
7.2 Possibili sviluppi futuri . . . 47
7.3 Considerazioni finali . . . 48

A Uso del prototipo **49**
A.1 Codice sorgente . . . 49
A.2 Requisiti e compilazione . . . 49
A.3 Ambiente di sviluppo . . . 50
A.4 Utilizzo . . . 50

Capitolo 1

Introduzione

1.1 Contesto

La sicurezza nei sistemi Linux si basa principalmente sulla prevenzione tramite meccanismi di permessi o configurazioni che permettono di limitare l'accesso alle risorse. Questo approccio non consente però di conoscere ciò che accade durante l'esecuzione del sistema. Basta quindi un errore di configurazione o un processo che ottiene erroneamente più privilegi di quelli che dovrebbe a causare potenzialmente problemi di sicurezza senza che nessuno possa accorgersene in tempo reale. Per questo motivo sono nati nel tempo strumenti che affiancano la prevenzione e rendono possibile l'osservazione a runtime delle operazioni svolte dai processi, così da poter individuare e segnalare eventuali operazioni sospette, sebbene permesse dal sistema.

Un tipico esempio di risorse sensibili sono i file il cui contenuto garantisce la sicurezza e l'integrità dell'intero sistema. Basti pensare a file come `/etc/shadow`, che contiene gli hash delle password degli utenti, o a `/etc/sudoers`, che stabilisce chi può ottenere i privilegi di amministratore. Una violazione di questi file può portare a un'escalation di privilegi e alla conseguente perdita di controllo del sistema. Avere degli strumenti in grado di osservare tutte le operazioni che coinvolgono risorse sensibili aiuta quindi a individuare un eventuale attacco o a ricostruirne l'avvenimento a posteriori.

Osservare queste operazioni a livello kernel ha il vantaggio che ogni forma di coinvolgimento di una risorsa sensibile deve passare da esso e non è quindi possibile aggirare facilmente il monitoraggio. A tale scopo risulta vantaggioso usare eBPF, che permette di estendere il kernel senza scrivere un modulo, caricando programmi che vengono verificati prima dell'esecuzione e invocati in specifici punti del nucleo [1]. Maggiori dettagli riguardo a questa tecnologia sono forniti nel Capitolo 2.

1.2 Strumenti esistenti

Proprio per le motivazioni sopra citate, negli ultimi anni sono nati molti strumenti che sfruttano eBPF per osservare, segnalare e in certi casi anche intervenire in caso

di comportamenti sospetti in un sistema Linux.

Uno dei più diffusi è Falco, creato da Sysdig e oggi progetto della Cloud Native Computing Foundation, che osserva gli eventi del kernel, come le chiamate di sistema, e li confronta con un insieme di regole definite dall'utente. Si tratta di uno strumento pensato per ambienti in cui si fa largo uso di container, che integra negli eventi le informazioni provenienti dal runtime dei container e da Kubernetes [2]. Un altro esempio è Tetragon, sviluppato all'interno del progetto Cilium, che oltre a osservare gli eventi permette anche di intervenire sulle operazioni [3].

Il lavoro presentato non ha né l'obiettivo né la presunzione di costruire un'alternativa a questi strumenti, ma ne prende l'intuizione e, attraverso un prototipo semplificato che si concentra solo sul tracciamento degli accessi a file, analizza un aspetto che una regola di monitoraggio dà per scontato, ovvero come sia possibile tenere traccia di una risorsa in modo efficace.

1.3 Obiettivo del lavoro

L'idea iniziale era la realizzazione di un monitor simile a Falco, in grado di segnalare eventi sospetti nel sistema anche in presenza di container, eventualmente estendibile in modo da consentire il blocco di tali operazioni in attesa di una revisione da parte dell'amministratore. Il punto di partenza è stato il monitoraggio e la segnalazione limitata agli accessi a file, realizzando un programma eBPF agganciato all'hook LSM invocato all'apertura di un file.

Il problema principale di questo tipo di monitoraggio non è però la raccolta degli eventi, ma il riconoscimento di un file da sorvegliare. Un file può essere indicato attraverso il suo path, che è il riferimento usato dall'utente nella configurazione, oppure attraverso la coppia device e inode, che individua l'oggetto del filesystem su cui il kernel lavora, ma nessuna delle due informazioni è sufficiente da sola. Il path non riconosce gli accessi eseguiti con un altro nome, ad esempio tramite un hard link, e all'interno di un container, che ha un mount namespace diverso da quello dell'host, può indicare un oggetto diverso. La coppia device e inode invece smette di corrispondere al file quando questo viene sostituito, come accade al salvataggio con editor come vim, e il suo numero può essere riassegnato a un file estraneo. Il centro della tesi è quindi come si possa identificare in modo affidabile, e senza perderne le tracce, una risorsa che l'utente vuole proteggere.

1.4 Metodo

Il lavoro è stato sviluppato per miglioramenti incrementali di un prototipo e questa relazione ripercorre il medesimo percorso. Ogni versione è stata messa alla prova con alcuni test che hanno messo in luce le criticità ma anche le caratteristiche e il funzionamento del kernel. L'analisi dei risultati ha portato a rivedere diverse volte il modo in cui identificare un file e la struttura dello strumento progettato.

Il codice del prototipo è disponibile in un repository pubblico [4], in cui ogni versione descritta nei capitoli successivi è contrassegnata da un tag. La corrispondenza tra tag e capitoli è riportata nell'Appendice A.

Il risultato di questo percorso non è quindi uno strumento pronto all'uso, ma un modello per l'identificazione della risorsa verificato sperimentalmente, accompagnato da una serie di casi che mostrano in quali modi un file può sfuggire al monitoraggio e da alcune questioni che restano aperte.

Capitolo 2

eBPF e libbpf

Questo capitolo presenta gli strumenti e i concetti utilizzati durante tutto il lavoro e illustra la struttura comune a tutte le versioni del prototipo.

2.1 eBPF

Come anticipato, il progetto si basa su eBPF, una tecnologia del kernel Linux che permette di eseguire al suo interno dei programmi forniti da user space, senza doverne modificare il codice sorgente o caricare un modulo. Questa tecnologia, in continua evoluzione, viene usata in diversi campi come il networking, il tracing, l'osservabilità e la sicurezza [1]. Un programma eBPF viene scritto tipicamente in un sottoinsieme del C e compilato con clang in bytecode eBPF. Il bytecode viene poi passato al kernel attraverso la system call `bpf()`, e il kernel prima di accettarlo verifica che non possa comprometterne la stabilità. Una volta caricato, il programma può essere agganciato a un punto del kernel ed è eseguito ogni volta che si verifica l'evento corrispondente. Questi punti possono essere ad esempio i tracepoint, cioè punti di osservazione predefiniti nel codice del kernel come l'ingresso di una system call, oppure gli hook LSM descritti più avanti.

Un'applicazione basata su eBPF è composta tipicamente da due programmi distinti. Il primo è il programma eBPF vero e proprio, che viene eseguito nel kernel e reagisce agli eventi a cui è agganciato, mentre il secondo è un normale programma in user space, che chiede al kernel di caricare e agganciare il programma eBPF, gli fornisce la configurazione e ne raccoglie i risultati. I due programmi sono legati fra loro e per comunicare usano strutture dati condivise, le mappe, descritte nella sezione successiva.

2.2 Verifier, funzioni helper e mappe

Il verifier è un componente del kernel che si occupa dell'analisi di un programma eBPF, controllandone il flusso e simulandone l'esecuzione lungo tutti i percorsi possibili così da poter individuare a priori eventuali problemi che potrebbero compromettere

la stabilità del kernel. In questo modo il verifier garantisce che il programma termini, che gli accessi alla memoria avvengano solo attraverso puntatori validi e che il valore restituito da operazioni che possono fallire, come la ricerca in una mappa, venga controllato prima di essere usato. Tra i diversi controlli c'è anche la dimensione dello stack, che nei programmi eBPF è limitata a 512 byte, un limite che è stato necessario aggirare durante lo sviluppo del prototipo. Una volta superata la verifica, il bytecode viene di norma tradotto dal compilatore JIT del kernel nel codice macchina della CPU in uso.

Un programma eBPF non può chiamare liberamente le funzioni del kernel, ma può usare gli helper, che costituiscono un'interfaccia stabile e permettono ad esempio di leggere l'identificativo del processo corrente o di accedere alle mappe [5]. Oltre agli helper possono essere usate le kfunc, funzioni del kernel esposte ai programmi eBPF che invece non hanno un'interfaccia stabile e possono cambiare da una versione del kernel all'altra.

Le mappe sono strutture dati chiave-valore che risiedono nel kernel e sono accessibili sia dai programmi eBPF sia da user space, per cui costituiscono il mezzo di comunicazione tra le due parti e il modo per conservare uno stato tra esecuzioni diverse [1]. Il prototipo ne usa tre tipi, le tabelle hash (`BPF_MAP_TYPE_HASH`) per la configurazione e lo stato, un array con una copia per ogni CPU (`BPF_MAP_TYPE_PERCPU_ARRAY`) come memoria di appoggio, vista la dimensione limitata dello stack, e un ring buffer (`BPF_MAP_TYPE_RINGBUF`) per inviare gli eventi a user space [6].

2.3 Linux Security Modules

Il framework LSM (Linux Security Modules) permette di creare moduli di sicurezza e di aggiungere controlli alle operazioni del kernel. Nei punti in cui il kernel prende decisioni rilevanti per la sicurezza, come l'apertura di un file o la cancellazione di un nome, sono posizionati degli hook, a cui ogni modulo, come SELinux o AppArmor, collega i propri controlli, e più moduli possono essere attivi contemporaneamente [7].

Dal kernel 5.7 è disponibile anche il modulo BPF-LSM, che esegue i programmi eBPF agganciati agli hook [1]. In questo modo è possibile realizzare politiche di controllo e di osservabilità degli accessi semplicemente caricando un programma, senza dover scrivere un nuovo modulo. Per usarlo il kernel deve essere compilato con l'opzione `CONFIG_BPF_LSM` e `bpf` deve comparire tra i moduli di sicurezza attivi.

Un programma LSM riceve gli argomenti dell'hook e può stabilire l'esito dell'operazione, ad esempio impedendola, anche se il prototipo si limita a segnalare gli accessi [8]. Rispetto a un tracepoint sulla system call di apertura, l'hook LSM `file_open` ha inoltre il vantaggio di essere invocato quando il kernel ha già risolto il percorso richiesto e creato la `struct file`, per cui il programma lavora direttamente sull'oggetto che sta per essere aperto e non sulla stringa passata dal processo.

2.4 BTF e CO-RE

I programmi eBPF hanno la necessità di leggere campi di strutture del kernel, che però non sono sempre identiche tra versioni differenti. Per descrivere queste strutture il kernel espone, nel file `/sys/kernel/btf/vmlinux`, la descrizione dei propri tipi in formato BTF (BPF Type Format), da cui con `bpftool` è possibile generare l'header `vmlinux.h`, che contiene le definizioni di tutti i tipi del kernel in esecuzione.

Queste definizioni però valgono solo per il kernel da cui sono state generate. Per rendere un programma portabile tra versioni diverse del kernel è stato quindi definito l'approccio CO-RE (Compile Once, Run Everywhere), che sfrutta le stesse informazioni BTF. Il programma viene compilato registrando, per ogni accesso a un campo, quale campo di quale struttura si vuole leggere, e al momento del caricamento libbpf confronta queste informazioni con il BTF del kernel in esecuzione e corregge gli offset di conseguenza [9]. Nel codice questo si traduce nell'uso di macro come `BPF_CORE_READ`.

2.5 libbpf

libbpf è la libreria di riferimento per caricare e gestire programmi eBPF da user space [10]. A partire dal file oggetto prodotto da clang, `bpftool` può generare uno skeleton, cioè un header che contiene il programma compilato e delle funzioni specifiche per aprirlo, caricarlo nel kernel, agganciarlo agli hook e distruggerlo, oltre a dei riferimenti diretti alle mappe e ai programmi che contiene. Il progetto è organizzato a partire da libbpf-bootstrap [11], che fornisce un Makefile per compilare il programma eBPF, generare lo skeleton e compilare il programma in user space. La libreria libbpf è inclusa come submodule git e compilata staticamente, in modo che il prototipo non dipenda dalla versione installata nel sistema.

2.6 Struttura del prototipo

Tutte le versioni del prototipo condividono la stessa struttura, basata sui due programmi descritti all'inizio del capitolo. Il programma in user space riceve come argomenti i file da sorvegliare e, per ciascuno, se segnalare le letture, le scritture o entrambe. Dopo aver caricato il programma eBPF, inserisce nelle mappe hash la configurazione, cioè le informazioni necessarie a riconoscere ogni file, e aggancia il programma agli hook. Da quel momento il programma eBPF viene invocato a ogni apertura di un file e, a partire dalla versione descritta nel Capitolo 5, anche alle operazioni che modificano i nomi dei file. A ogni invocazione confronta il file coinvolto con la configurazione e, se il file è sorvegliato, invia un evento attraverso il ring buffer, che il programma in user space legge e stampa. Questo schema è riassunto nella Figura 2.1. Il modo in cui un file viene riconosciuto è proprio ciò che cambia tra le diverse versioni ed è descritto nei capitoli successivi.

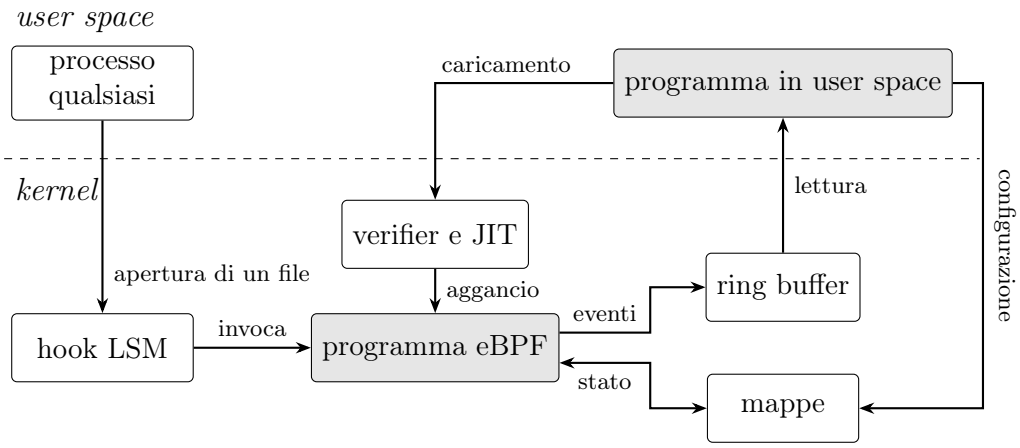


Figura 2.1: I due programmi che compongono il prototipo, evidenziati in grigio. Il programma in user space chiede al kernel di caricare il programma eBPF, che dopo la verifica viene agganciato agli hook LSM, e i due comunicano attraverso le mappe e il ring buffer.

Ogni evento contiene le informazioni sul file e sul processo che ha eseguito l'operazione, che a partire dalla seconda versione comprendono anche il path, i namespace e il cgroup.

2.7 Ambiente di sviluppo

Il prototipo è stato sviluppato e testato in una macchina virtuale con Ubuntu 26.04 LTS e kernel Linux 7.0.0, configurato con il supporto a BPF-LSM e a BTF. I dettagli sull'ambiente e sui requisiti per compilare ed eseguire il prototipo sono riportati nell'Appendice A.

Capitolo 3

Identificare un file

Prima di entrare nel merito del lavoro vero e proprio è necessario descrivere come il kernel rappresenta i file e i loro nomi, perché è da qui che nasce il problema di riconoscere in modo affidabile il file che l'utente vuole sorvegliare.

3.1 Il filesystem visto dal kernel

Con filesystem si intende l'organizzazione di file e directory su un dispositivo di memorizzazione, come un disco o un'unità USB; esistono anche filesystem che non risiedono in un dispositivo fisico ma che sono mantenuti dal kernel in memoria. Ogni filesystem gestisce i propri file numerandoli in modo indipendente, quindi due file che si trovano su filesystem diversi possono avere lo stesso numero di inode, cioè il numero con cui il filesystem identifica ciascun file, pur essendo oggetti completamente distinti. Per questo motivo il kernel associa a ogni filesystem anche un identificativo, che per i filesystem su disco coincide con il numero del dispositivo su cui si trovano, composto da un numero maggiore, che indica il tipo di dispositivo, e da un numero minore, che indica la specifica istanza [12], mentre per i filesystem che non risiedono su un dispositivo viene assegnato dal kernel stesso. Nel seguito questo identificativo viene chiamato semplicemente device, per cui dire che due file hanno device diversi significa che si trovano su filesystem diversi.

In Linux esiste un unico albero di directory, e un filesystem diventa accessibile solo quando viene montato, cioè collegato a un punto dell'albero, di norma una directory, che prende il nome di punto di montaggio [13]. Lo stesso filesystem può anche essere montato in più punti, per cui gli stessi file possono comparire in posizioni diverse dell'albero.

I diversi filesystem sono gestiti dal Virtual File System (VFS), un componente del kernel che offre un'interfaccia comune indipendentemente dal filesystem su cui si trovano i dati. Il VFS descrive file e directory attraverso alcune strutture definite negli header del kernel, le cui relazioni sono mostrate in modo sintetico e schematico nella Figura 3.1.

L'inode (`struct inode`) rappresenta il file vero e proprio e ne contiene i metadati, tra cui il numero di inode (`i_ino`), il numero di nomi che fanno riferimento al file (`i_nlink`) e il puntatore al superblock del filesystem a cui appartiene (`i_sb`). Il superblock (`struct super_block`) descrive un filesystem montato e contiene nel campo `s_dev` il suo device. Se lo stesso filesystem viene montato in più punti, i diversi montaggi condividono lo stesso superblock, per cui i suoi file mantengono lo stesso device indipendentemente dal percorso con cui vengono raggiunti. L'inode non contiene però il nome del file.

Il nome è rappresentato dalla dentry (`struct dentry`), che collega un nome (`d_name`) a un inode (`d_inode`) e che punta alla dentry della directory padre (`d_parent`). Risalendo i padri a partire da una dentry si arriva alla radice del filesystem, ma non si sa dove questo sia montato nell'albero complessivo, informazione che si trova invece nella `struct vfsmount`. Per questo il kernel rappresenta un percorso con la `struct path`, che contiene sia il montaggio sia la dentry.

Infine la `struct file` rappresenta un file aperto da un processo e contiene la `struct path` con cui il file è stato raggiunto (`f_path`), l'inode (`f_inode`) e i flag di apertura (`f_flags`). È questa la struttura che l'hook `file_open` riceve come argomento.

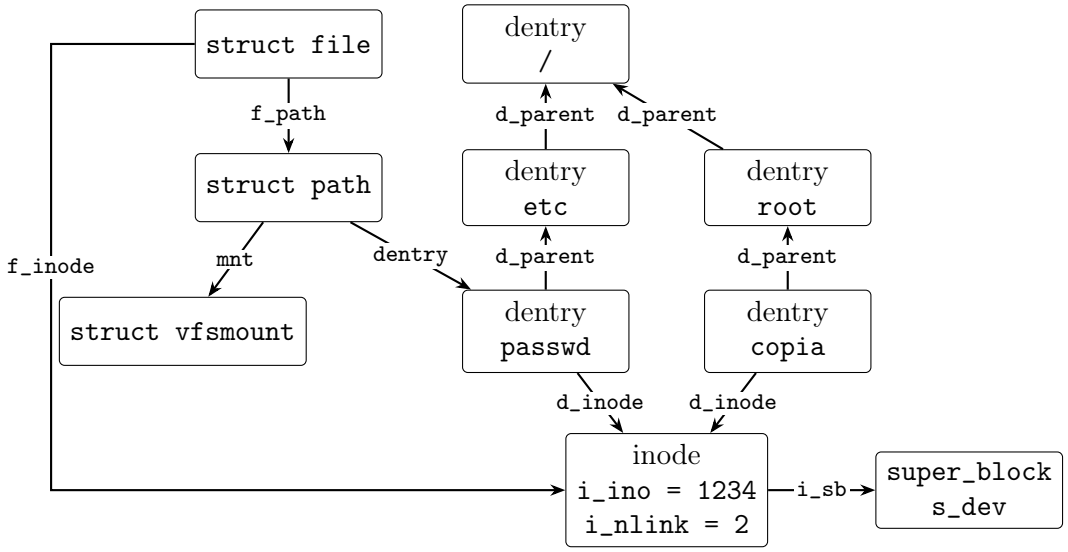


Figura 3.1: Strutture del VFS coinvolte nell'apertura di `/etc/passwd`, con un hard link `/root/copia` allo stesso file. I due nomi sono dentry diverse che puntano allo stesso inode, il cui campo `i_nlink` vale per questo 2.

3.2 Percorso e oggetto

Dalle strutture del VFS emerge che il nome di un file, cioè il path, e l'oggetto a cui corrisponde, cioè l'inode, sono due entità distinte, legate tra loro dalla catena

di dentry e dai montaggi attraversati durante la risoluzione del path. Un file può quindi essere identificato in due modi che non sono equivalenti.

Il numero di inode è univoco solo all'interno di un filesystem, per cui da solo non basta a identificare un oggetto nel sistema. Se ad esempio il filesystem principale si trova sulla partizione `sda1`, con device 8:1, e una seconda partizione `sdb1`, con device 8:17, è montata in `/mnt/dati`, il file `/etc/passwd` e il file `/mnt/dati/foto.jpg` potrebbero avere entrambi il numero di inode 1234, ma le coppie (8:1, 1234) e (8:17, 1234) restano diverse. La coppia (device, inode) identifica quindi in modo univoco un oggetto, e nel seguito viene indicata semplicemente come inode quando non ci sono ambiguità.

Questa identità vale però solo finché l'oggetto esiste, perché quando l'ultimo nome di un file viene rimosso e nessun processo lo tiene aperto il file viene cancellato [14], e il suo numero di inode può essere assegnato a un nuovo file, come mostrano i test del Capitolo 4.

D'altra parte un oggetto non è necessariamente legato a un unico nome, perché un hard link è una seconda dentry che punta allo stesso inode, come `/root/copia` nella Figura 3.1, per cui lo stesso oggetto è raggiungibile con nomi diversi. Un hard link può essere creato solo all'interno dello stesso filesystem montato [15]. Un link simbolico è invece un file a sé che contiene un percorso, e durante l'apertura il kernel lo sostituisce con il percorso che contiene, per cui chi lo apre normalmente apre in realtà il file a cui punta.

Le operazioni comuni sul filesystem agiscono in modo diverso su nomi e oggetti. Un rename, che può avvenire solo all'interno dello stesso filesystem montato, sposta solo il nome e lascia invariato l'inode e gli altri eventuali hard link [16], e se a essere rinominata è una directory lungo il percorso cambia il path di tutti i file che contiene senza che questi vengano modificati. La rimozione di un nome invece distrugge l'oggetto solo se era l'ultimo. Inoltre alcuni programmi, come l'editor vim nella configurazione predefinita usata nei test del Capitolo 4, non modificano un file riscrivendone il contenuto, ma ne creano uno nuovo che prende il posto del precedente, per cui dopo il salvataggio il nome è lo stesso ma l'oggetto è diverso.

Infine il path non ha un significato assoluto, perché viene risolto a partire dalla directory radice del processo e attraverso i montaggi che il processo vede [17, 18]. Processi con una vista diversa del filesystem, come quelli eseguiti all'interno di un container, possono quindi ottenere oggetti diversi a partire dallo stesso path, un aspetto che viene approfondito nel Capitolo 6.

3.3 Semantiche possibili per una policy

Quando l'utente indica un file da sorvegliare non è quindi scontato cosa debba essere protetto, e si possono individuare tre semantiche diverse.

Proteggere il path significa sorvegliare qualunque oggetto si trovi in quel momento sotto quel nome. Le sostituzioni del file non fanno perdere la risorsa, ma basta

raggiungere lo stesso contenuto con un altro nome, ad esempio con un hard link, per non essere riconosciuti.

Proteggere l'oggetto significa sorvegliare uno specifico inode indipendentemente dal nome usato. Gli accessi tramite hard link o dopo un rename vengono riconosciuti, ma una sostituzione fa perdere la risorsa e il riciclo del numero di inode può attribuirle accessi a file estranei.

La terza semantica consiste nel mantenere dinamicamente il legame tra i due, usando il path per indicare cosa proteggere e l'inode per identificare l'oggetto che in quel momento corrisponde al path, e aggiornando il legame quando il filesystem cambia.

Le prime due semantiche corrispondono ai criteri confrontati nel Capitolo 4, mentre la terza è quella adottata a partire dal Capitolo 5.

Capitolo 4

Strategie di identificazione a confronto

Questo capitolo descrive le prime due versioni del monitor degli accessi a file. La prima identifica il file solo attraverso la coppia (device, inode), mentre la seconda è stata costruita per confrontare sperimentalmente i due modi di identificare una risorsa descritti nel Capitolo 3, la coppia (device, inode) e il path. Ogni risultato ottenuto nelle fasi di seguito descritte ha indirizzato il percorso e le scelte, e i test mostrano concretamente quali operazioni fanno perdere traccia del file con ciascuno dei due criteri.

In tutte le sezioni successive l'attenzione viene concentrata sul programma eBPF e su come vengono identificati i file da sorvegliare. Passa in secondo piano il programma in user space, che si occupa di far caricare il programma eBPF nel kernel, della configurazione e della ricezione e stampa degli eventi.

4.1 Architettura iniziale

Sin dall'inizio il programma è stato pensato in modo che l'utilizzatore potesse specificare i path dei file da osservare e se tracciare solo le aperture in lettura, in scrittura o entrambe. Il primo prototipo è stato costruito a partire dagli esempi di libbpf-bootstrap [11] e si aggancia all'hook LSM `file_open` così da tracciare le aperture dei file. Il kernel lo invoca a ogni apertura dopo la risoluzione del percorso e prima dell'effettivo completamento dell'operazione. Questa versione corrisponde al tag `v1-inode` del repository, riportato nella Tabella A.1.

La configurazione viene passata dal programma in user space inserendo come chiave la coppia (device, inode), ricavata con `stat()`, in una mappa di tipo `BPF_MAP_TYPE_HASH`. A ogni chiave è associata la modalità di apertura da intercettare.

In questo primo approccio il programma nel kernel è stato mantenuto semplice, e a ogni apertura di un file viene eseguita una lookup nell'hash map sulla coppia (device, inode), per verificare se l'apertura deve essere notificata tramite il ring buffer al processo in user space. A livello kernel viene anche individuata la modalità

di apertura del file, così da poter filtrare gli eventi in base alle opzioni specificate dall'utente. Il Listing 4.1 riporta la parte del programma che legge la coppia (device, inode) dalla `struct file`, esegue la lookup e filtra l'evento in base alla modalità di apertura.

```
43  if (BPF_CORE_READ_INTO(&ino, file, f_inode, i_ino))
44    return 0;
45  if (BPF_CORE_READ_INTO(&dev, file, f_inode, i_sb, s_dev))
46    return 0;
47
48  __builtin_memset(&key, 0, sizeof(key));
49  key.ino = ino;
50  key.dev = dev;
51
52  r = bpf_map_lookup_elem(&watched, &key);
53  if (!r)
54    return 0;
55
56  if (BPF_CORE_READ_INTO(&flags, file, f_flags))
57    return 0;
58
59  mode = flags & O_ACCMODE;
60
61  if (mode == O_RDONLY && !r->on_read)
62    return 0;
63  if (mode == O_WRONLY && !r->on_write)
64    return 0;
65  if (mode == O_RDWR && !r->on_read && !r->on_write)
66    return 0;
```

Listing 4.1: `handle_file_open`

4.2 Il problema dell'editor

Nonostante il programma tracciasse correttamente le aperture, i primi test hanno riprodotto il limite del criterio per inode descritto nel Capitolo 3, a partire dalla modifica dei file con editor come vim. Al salvataggio questo strumento non sovrascrive il contenuto del file, ma sposta l'originale in una copia momentanea, scrive il contenuto in un nuovo file con lo stesso nome e infine cancella la copia. Il nome e il path della risorsa restano gli stessi, ma il file salvato è un oggetto diverso da quello di partenza, con un altro inode, per cui le aperture successive alla modifica non vengono più monitorate. Lo stesso accade se il file viene cancellato e ricreato con lo stesso nome in quanto il nuovo file è un oggetto diverso e il programma non ha più alcun riferimento per riconoscerlo come risorsa sorvegliata.

Individuare un file solo attraverso l'inode non è quindi sufficiente, perché operazioni apparentemente innocue sostituiscono l'oggetto che corrisponde al path e fanno perdere traccia del file. L'utente invece, una volta specificato un path, si aspetta di poter tracciare il file indipendentemente dalle operazioni che lo coinvolgono. Prima di poter scegliere una strategia è stato necessario confrontare sperimentalmente i due criteri.

4.3 Aggiunta del criterio per path

Per affrontare il problema individuato è stata aggiunta una seconda mappa, in cui il programma in user space salva il path specificato dall'utente.

Sono quindi presenti due mappe indipendenti, `watched_inode`, con chiave la coppia (device, inode) come nella versione precedente, e `watched_path`, con chiave il percorso canonico. A ogni apertura il programma eBPF consulta entrambe e riporta nell'evento quale ha trovato corrispondenza (`ino`, `path` o `both`), così da rendere possibile il confronto, scopo principale di questa configurazione. In questa versione l'evento è stato inoltre esteso con alcune informazioni sul contesto del processo, come loginuid, namespace e cgroup, che verranno utilizzate più avanti. Questa versione corrisponde al tag `v2-inode-path`.

4.3.1 Risoluzione del path

Per confrontare i due criteri serve conoscere, al momento dell'apertura, il percorso del file aperto. La `struct file` non contiene il percorso come stringa, ma una `struct path`. Ricostruire la stringa richiede di risalire la catena delle dentry fino alla radice attraversando i punti di montaggio, e farlo a mano in un programma eBPF è complesso per via dei vincoli del verifier.

È stato quindi usato l'helper `bpf_d_path()`, che riceve un puntatore alla `struct path` e scrive il percorso assoluto in un buffer, restituendone la lunghezza con il terminatore compreso [5]. Per avere un confronto coerente, anche in user space il percorso indicato dall'utente è stato preso in forma canonica risolvendolo con `realpath()`, che elimina i riferimenti relativi e risolve eventuali link simbolici [19].

4.3.2 Vincoli imposti dal verifier e memoria di appoggio

L'introduzione del path ha portato i primi problemi legati ai vincoli dell'ambiente eBPF.

Il kernel prevede path di lunghezza al massimo di `PATH_MAX` caratteri, cioè 4096 byte, mentre lo stack di un programma eBPF, come già discusso nel Capitolo 2, è limitato a 512 byte. Per questo è stata introdotta una mappa di tipo `BPF_MAP_TYPE_PERCPU_ARRAY` con un solo elemento, usata come memoria di appoggio. Il fatto che la mappa sia per-CPU garantisce che esecuzioni concorrenti del programma su CPU diverse non condividano lo stesso buffer [1].

Il buffer di appoggio serve anche perché l'helper `bpf_d_path()` scrive il percorso all'inizio del buffer, ma dopo il terminatore non vi è garanzia che i byte siano tutti a zero, mentre la lookup nella mappa hash confronta tutti i byte della chiave. Prima della lookup il path viene quindi copiato fino al terminatore in un secondo buffer azzerato.

Con 4096 byte però l'azzeramento con `__builtin_memset` non viene compilato, perché il compilatore genera una chiamata alla funzione `memset`, che nei programmi eBPF non è disponibile. Per proseguire con il confronto si è scelto di limitare la

lunghezza massima del percorso a 256 byte, che rimane un limite del prototipo e non del problema. Una possibile alternativa sarebbe usare come chiave un hash del path, con il costo di dover gestire le collisioni.

4.4 Test per il confronto

Per poter valutare le due strategie di riconoscimento sono stati effettuati diversi test, che sono consistiti in alcune operazioni applicate a un file monitorato dal programma. Dopo ogni operazione il file è stato aperto con il comando `cat` e il risultato è stato valutato osservando l'evento generato da questa apertura. Ogni caso è stato eseguito in modo isolato, ricreando i file di prova e riavviando il programma, così che nessuno stato passasse da un caso all'altro.

È stato fondamentale il campo MATCH dell'evento, che riporta se il file è stato identificato grazie alla coppia (device, inode), con il valore `ino`, tramite il path, con il valore `path`, o con entrambi, con il valore `both`. Questo dato consente di capire, per ogni test, quale criterio permette di non perdere il tracciamento del file.

Ogni test modifica in modo diverso il rapporto tra il nome del file e l'oggetto che gli corrisponde e i casi si possono dividere in tre gruppi. Oltre a una semplice apertura, utile per testare il funzionamento nel caso standard, il primo gruppo riguarda i casi in cui il nome resta lo stesso mentre l'oggetto cambia. Il file è stato modificato con vim in modo da riprodurre il problema riscontrato nella prima versione, è stato sostituito con un altro file tramite il comando `mv` ed è stato cancellato e ricreato con lo stesso nome. Questi casi mettono alla prova il criterio per inode.

Il secondo gruppo riguarda invece i casi in cui l'oggetto resta lo stesso mentre cambia, o si aggiunge, il nome con cui viene raggiunto. Il file è stato aperto tramite un hard link ed è stata rinominata la directory padre. Questi casi vogliono testare il criterio per path.

L'ultimo gruppo riguarda i symlink, usati sia per aprire il file sia per configurare il monitor, in modo da verificare che la risoluzione del nome sia coerente tra user space e kernel.

Per ogni caso si è inoltre verificato se il numero di inode del file cambiasse a seguito dell'operazione.

4.5 Risultati

I risultati dei test sono riassunti nella Tabella 4.1.

Tabella 4.1: Confronto tra il criterio per (device, inode) e il criterio per path. Il match si riferisce all'apertura con `cat` eseguita dopo ogni operazione.

Caso	Inode cambia	Match	Note
Apertura semplice	no	`both`	Comportamento atteso.
Modifica con vim	sì	`path`	Il numero del vecchio inode viene riassegnato a un altro file, che genera un falso positivo per inode.
Sostituzione con `mv`	sì	`path`	La sostituzione in sé non genera eventi, perché non passa da `file_open`.
Unlink e ricreazione	dipende	`path` o `both`	L'inode liberato può essere assegnato a un file diverso, quindi il match per inode non è deterministico.
Apertura tramite hard link	no	`ino`	Il path riportato è quello dell'hard link, che il criterio per path non riconosce.
Directory padre rinominata	no	`ino`	Il path riportato è quello nuovo, che non corrisponde più alla configurazione.
Apertura tramite symlink	no	`both`	Il kernel risolve il link prima di `file_open` e il path riportato è quello del target.
Monitor configurato su un symlink	no	`both`	`realpath()` risolve il link in fase di configurazione, in modo coerente con il kernel.

Con l'utilizzo di vim, dopo il salvataggio il match per path continua a funzionare, quello per inode invece no. Il programma mostra però un altro effetto: il numero dell'inode monitorato viene riassegnato dal filesystem a un file temporaneo creato da vim stesso, `.viminfo.tmp`, e l'apertura di quel file genera un evento con MATCH `ino` attribuito al file sorvegliato. L'identificazione per inode quindi non solo perde il bersaglio dopo una sostituzione, ma può produrre falsi positivi su file estranei che ne ereditano il numero.

La sostituzione con `mv` porta allo stesso risultato, perché anche in questo caso la `cat` successiva viene riconosciuta solo tramite il path. In più l'operazione in sé non comporta nessuna apertura del file e quindi non genera alcun evento, per cui la sostituzione è invisibile a un programma agganciato solo a `file_open`.

Dopo la rimozione, l'inode liberato può essere riassegnato dal filesystem al file successivo che viene creato. Se questo è il file ricreato con lo stesso nome, il match per inode continua a funzionare, ma per coincidenza e non perché stia identificando la stessa risorsa. Se invece a ereditare l'inode è un altro file, le sue aperture vengono

segnalate come accessi al file sorvegliato, portando quindi a un comportamento sbagliato.

L'apertura attraverso un hard link viene riconosciuta dal criterio per inode ma non da quello per path, che riceve il percorso effettivamente usato. Chi può creare un hard link a una risorsa sorvegliata può quindi accedervi aggirando una policy basata sul solo percorso.

Nel caso della directory padre rinominata il file non viene modificato in alcun modo, ma cambia la sua posizione nel filesystem. Solo il criterio per inode continua a consentire il monitoraggio dell'apertura.

Il symlink viene risolto correttamente sia da `realpath()` in user space, quando si costruisce la policy, sia dal kernel prima che l'hook venga invocato. Le due risoluzioni producono lo stesso percorso canonico, quindi il matching funziona indipendentemente da quale nome venga usato per configurare o per aprire.

4.6 Considerazioni

I risultati confermano la distinzione descritta nel Capitolo 3, per cui i due criteri, path e (device, inode), non sono due modi di fare la stessa cosa, ma identificano concetti diversi. Il path identifica un nome all'interno dell'albero dei file, indipendentemente dall'oggetto che gli corrisponde, mentre la coppia (device, inode) identifica un oggetto indipendentemente dal nome con cui viene raggiunto, ma solo finché questo esiste. Per entrambe le strategie esiste almeno un'operazione che fa perdere traccia del file e il caso più grave è quello dell'inode riciclato, perché oltre a perdere la risorsa porta ad attribuirle gli accessi a un file diverso.

Una possibile soluzione consiste quindi nell'unire i due criteri, creando un programma in grado di riconoscere l'apertura del file in diversi casi. Per tenerli allineati però non basta osservare le aperture, ma è necessario osservare anche le operazioni che modificano il legame tra nomi e oggetti, come rename, unlink e link.

Capitolo 5

Strategia ibrida

Di seguito viene descritta la strategia ibrida, che implementa la terza semantica descritta nel Capitolo 3, cioè il mantenimento dinamico del legame tra path e oggetto, resa necessaria dai limiti dei singoli criteri emersi nel capitolo precedente. Questa versione corrisponde al tag `v3-hybrid` del repository. Vengono inoltre discussi i motivi e l'implementazione degli hook link, unlink e rename e infine vengono presentati i risultati dei test.

5.1 Il path come riferimento principale

Tra la coppia (device, inode) e il path è stato quest'ultimo a essere scelto come riferimento primario. È infatti attraverso il path che l'utente indica la risorsa da proteggere ed è quindi il percorso a definire cosa deve essere sorvegliato. La coppia (device, inode) rappresenta invece l'oggetto che in un determinato momento corrisponde al path. Rimane comunque fondamentale in diversi casi, come l'apertura tramite un hard link, perché permette di riconoscere l'oggetto anche quando viene raggiunto con un nome diverso, ma è un criterio subordinato al path specificato.

Per ottenere questo schema la mappa `watched_inode` della versione precedente è stata sostituita da una nuova mappa, `resolved_inode`, con un ruolo diverso. `watched_path` associa ora a ogni path sia le regole per il monitoraggio sia la coppia (device, inode) dell'oggetto che in quel momento corrisponde al path. `resolved_inode` fa invece il percorso inverso e associa a ogni coppia (device, inode) il path sorvegliato a cui l'oggetto è legato.

La seconda mappa serve principalmente quando un file viene aperto con un nome diverso da quello sorvegliato, perché in quel caso il programma ha a disposizione solo il path usato e la coppia (device, inode), e dato che una hash map può essere interrogata solo per chiave e non per valore, per risalire alla risorsa è necessaria una mappa che abbia come chiave proprio l'inode. Inoltre le due mappe non contengono sempre le stesse informazioni, perché `watched_path` tiene per ogni path un solo oggetto corrente, mentre `resolved_inode` può contenere anche oggetti che non corrispondono più al path ma che devono comunque essere sorvegliati, come nel caso

di un file sorvegliato che viene rinominato e sostituito da un nuovo file con lo stesso nome. La Figura 5.1 mostra il contenuto delle due mappe proprio in questo caso.

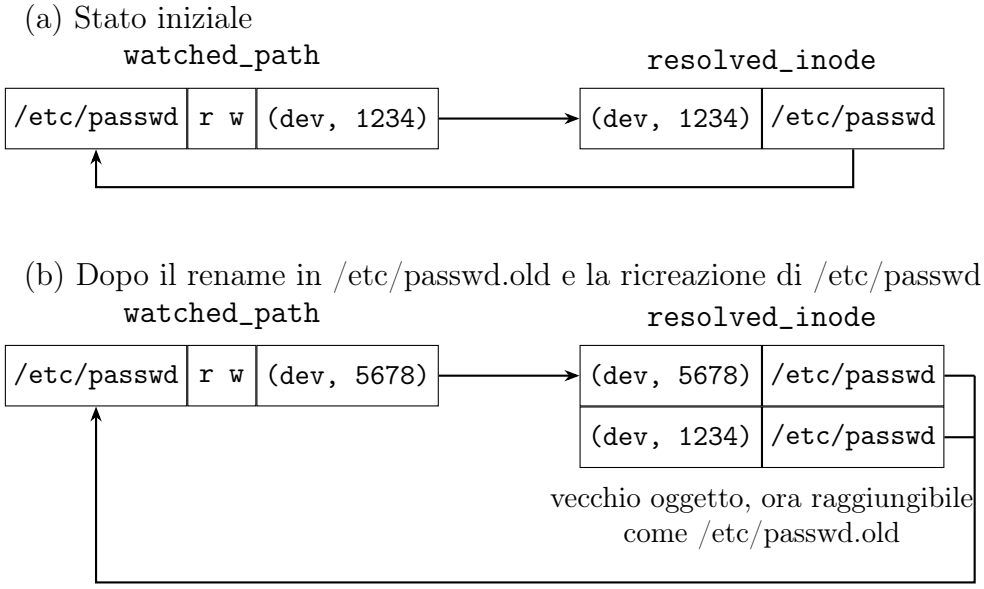


Figura 5.1: Contenuto delle due mappe prima e dopo il rename del file sorvegliato e la sua ricreazione

Mantenere aggiornate le due mappe intercettando solo `file_open` non è però possibile. Come mostrato nel capitolo precedente, una sostituzione con `mv` non comporta nessuna apertura del file e il programma se ne accorgerebbe quindi solo all'apertura successiva. Risulta necessario intercettare altri hook che, a livello kernel, mantengano aggiornate le mappe quando un file sorvegliato viene rinominato, cancellato o collegato con un hard link. Dato che questi hook sono comunque necessari, sono stati sfruttati anche per segnalare gli eventi al processo in user space, sempre tramite il ring buffer. Con questo nuovo approccio non solo le aperture vengono riconosciute in modo coerente, ma si hanno anche informazioni quando un file monitorato subisce una delle operazioni tracciate.

5.2 Riallineamento all'apertura

Con la nuova impostazione il programma agganciato a `file_open` consulta entrambe le mappe a ogni apertura. Il path del file aperto viene cercato in `watched_path`, mentre la coppia (device, inode) viene cercata in `resolved_inode`. Se il path non viene trovato ma l'inode sì, la regola da applicare viene recuperata da `watched_path` attraverso il path sorvegliato salvato in `resolved_inode`. In base alle corrispondenze trovate si possono presentare i casi riassunti nella Tabella 5.1. Rispetto al capitolo precedente cambia il significato del campo MATCH, dato che le due mappe non sono più sullo stesso piano. Il valore `path` indica ora che il percorso aperto è uno

di quelli specificati dall'utente, il valore `ino` che l'oggetto aperto è associato a una risorsa sorvegliata e il valore `both` che percorso e oggetto sono allineati.

Tabella 5.1: Comportamento di `file_open` in base alle corrispondenze trovate nelle due mappe

Match	Situazione	Azione del programma
`both`	Il path sorvegliato corrisponde ancora allo stesso oggetto.	L'apertura viene segnalata senza modificare le mappe.
`path`	L'oggetto associato al path è cambiato, ad esempio dopo una modifica con vim, una sostituzione con `mv` o una ricreazione.	L'inode salvato in `watched_path` viene rimosso da `resolved_inode` e sostituito con il nuovo, che diventa il valore corrente in `watched_path`. L'evento viene marcato come `replaced`.
`ino`	Lo stesso oggetto viene raggiunto con un altro nome, ad esempio tramite un hard link o dopo un rename.	L'apertura viene segnalata riportando sia il path usato sia il path sorvegliato a cui l'oggetto è associato.
nessuno	Il file non è sorvegliato.	Nessun evento.

Il secondo caso è quello che permette di non perdere traccia della risorsa dopo una sostituzione. Quando il path corrisponde ma l'inode aperto è diverso da quello salvato in `watched_path`, il programma considera il nuovo oggetto come la versione attuale della risorsa e aggiorna di conseguenza entrambe le mappe. Se invece il legame tra il path e il vecchio oggetto è già stato sciolto da uno degli hook, come nel caso della Figura 5.1, il vecchio oggetto non viene rimosso e resta sorvegliato in `resolved_inode`. In questo modo il path viene usato come riferimento per riallineare lo stato, mentre l'inode serve a riconoscere l'oggetto anche quando viene aperto con un nome diverso da quello sorvegliato. Il Listing 5.1 riporta la parte di `handle_file_open` che esegue le due ricerche e il riallineamento.

```
94  r = bpf_map_lookup_elem(&watched_path, &mem->clean_key);
95  if (r) {
96    by_path = 1;
97  }
98
99  cached_path = bpf_map_lookup_elem(&resolved_inode, &ikey);
100 if (cached_path) {
101   by_inode = 1;
102   if (!r) {
103     r = bpf_map_lookup_elem(&watched_path, cached_path);
104   }
105 }
106
107 if (!r) {
108   return 0;
```

```
109   }
110
111   if (by_path && (r->current_ikey.ino != ino || r->current_ikey.dev != dev)) {
112     inode_changed = 1;
113     bpf_map_delete_elem(&resolved_inode, &r->current_ikey);
114     bpf_map_update_elem(&resolved_inode, &ikey, &mem->clean_key, BPF_ANY);
115     r->current_ikey = ikey;
116   }
```

Listing 5.1: Ricerca nelle due mappe e riallineamento in `handle_file_open`

Il riallineamento però avviene solo al momento dell'apertura, quindi tra una sostituzione e l'apertura successiva le mappe possono contenere un inode che non corrisponde più al path. Per questo motivo sono necessari gli hook descritti nella sezione successiva, che mantengono aggiornate le mappe nel momento in cui il filesystem viene modificato.

In questa versione inoltre l'helper `bpf_d_path()` è stato sostituito con `bpf_path_d_path()`. Si tratta di una kfunc disponibile dalla versione 6.12 del kernel e utilizzabile solo dai programmi BPF-LSM, che viene indicata come la variante più sicura del vecchio helper e da preferire quando possibile, perché il verifier controlla che il puntatore alla `struct path` sia valido e in caso contrario rifiuta il programma. Il funzionamento è comunque lo stesso di `bpf_d_path()`, quindi la sostituzione non ha richiesto modifiche al resto del codice e restano validi i vincoli sul buffer descritti nel capitolo precedente.

5.3 Seguire le modifiche del filesystem

Come già detto, per mantenere aggiornate le mappe sono stati intercettati altri tre hook LSM: `path_unlink`, invocato quando un nome viene rimosso, `path_rename`, invocato quando un nome viene spostato, e `path_link`, invocato quando viene creato un hard link.

Oltre ad aggiornare le mappe, questi hook inviano un evento al processo in user space quando l'operazione coinvolge un path o un inode sorvegliato. Come per le aperture, ogni evento contiene le informazioni sul processo e su quale mappa ha trovato corrispondenza, insieme ai nomi coinvolti nell'operazione e al path sorvegliato a cui l'oggetto è associato. In particolare per l'unlink viene riportato il nome rimosso, per il rename il vecchio e il nuovo path e per il link il nuovo nome. In questa versione è stato inoltre aggiunto un timestamp a tutti gli eventi e la stampa lato user space è stata differenziata per ogni hook.

Per queste operazioni il framework LSM mette a disposizione anche gli hook `inode_unlink`, `inode_rename` e `inode_link`. Sono stati scelti gli hook `path_*` perché ricevono la `struct path` della directory, che, come visto nel Capitolo 3, oltre alla dentry contiene anche il punto di montaggio del filesystem e permette quindi di ricostruire il percorso assoluto con `bpf_path_d_path()`. Gli hook `inode_*` ricevono invece solo l'inode della directory e la dentry, da cui si può risalire fino alla radice del filesystem a cui appartiene il file ma non sapere dove questo è montato, e quindi non è

possibile ricavare il path completo da confrontare con quello sorvegliato. Un limite è però che gli hook `path_*` vengono invocati solo durante le system call, mentre gli hook `inode_*` vengono invocati dalle funzioni del VFS, che sono chiamate sia dalle system call sia direttamente dal kernel per operazioni eseguite internamente. Le operazioni che non passano da una system call non vengono quindi osservate. Inoltre gli hook `path_*` sono disponibili solo se il kernel è compilato con l'opzione `CONFIG_SECURITY_PATH`.

5.3.1 Comporre il path senza `struct file`

Un primo problema riscontrato è stato dovuto alle firme di questi hook, che a differenza di `file_open` non ricevono una `struct file`. `path_unlink` riceve la `struct path` della directory e la dentry del nome rimosso, `path_rename` riceve le stesse informazioni sia per la sorgente sia per la destinazione, mentre `path_link` riceve per la sorgente solo la dentry e per la destinazione la directory e la nuova dentry. Per ricavare il path del file sotto forma di stringa non è stato quindi possibile usare `bpf_path_d_path()` direttamente, ma si è dovuto comporlo ricavando il path della directory padre e aggiungendo il nome del file. Nel caso di `path_link` il file sorgente viene invece riconosciuto solo tramite la coppia (device, inode) in `resolved_inode`.

È importante sottolineare un altro limite legato alla discordanza tra i valori definiti dal kernel e quelli utilizzati in questo prototipo, come già visto nel capitolo precedente per la lunghezza del path. La lunghezza massima del nome di un file o di una directory, che nel kernel è di 255 caratteri, non è stata mantenuta e per gli stessi motivi visti in precedenza si è preferito adottare una semplificazione, limitandola a 63 caratteri. Di conseguenza anche il path della directory padre non può superare i 192 byte, cioè la lunghezza massima del path meno quella riservata al nome, altrimenti la composizione fallisce. Si tratta comunque di valori coerenti con la lunghezza massima definita per un path sorvegliabile anche se rimane un limite.

5.3.2 `path_unlink`

Intercettare `path_unlink` aveva come obiettivo sia notificare l'utente che aggiornare le mappe in caso di cancellazione del file.

In una prima implementazione intermedia di `handle_path_unlink`, non contrassegnata da un tag, il coinvolgimento di un file veniva valutato guardando solo l'inode. Il problema però è che un inode può avere più nomi. Rimuovendo per esempio un hard link, il programma credeva che stesse sparendo il file sorvegliato, lo segnalava nel ring buffer e scioglieva un'associazione ancora valida. Inoltre rimuoveva l'oggetto da `resolved_inode` anche se questo continuava a esistere con l'altro nome, e l'apertura successiva del file sorvegliato veniva marcata come sostituzione pur essendo lo stesso inode.

Per evitare che ciò accadesse è stato necessario ricostruire il nome che sta effettivamente sparendo e confrontarlo con il path sorvegliato, distinguendo così due situazioni. Se sparisce il nome contenuto in `watched_path` viene azzerato il legame

tra path e inode, perché è quel legame a non valere più, indipendentemente da cosa succede all'oggetto. Se invece viene rimosso l'ultimo nome dell'oggetto, cioè quando `i_nlink` è minore o uguale a 1, la voce viene rimossa da `resolved_inode`, perché il numero di inode verrà riciclato e potrebbe essere ereditato da un file estraneo. In questo modo si evitano i falsi positivi riscontrati nei test del capitolo precedente. Il Listing 5.2 mostra il confronto tra il nome rimosso e il path sorvegliato e il controllo sul numero di link dell'oggetto.

```
249   BPF_CORE_READ_INTO(&nlink, dentry, d_inode, i_nlink);
250   last_link = (nlink <= 1);
251
252   composed = (compose_path(&mem->dest_key, dir, dentry, mem->dir_path) == 0);
253   if (composed) {
254     is_watched_name = (__builtin_memcmp(&mem->dest_key, cached_path, sizeof(mem->dest_key)
      ) == 0);
255   } else {
256     is_watched_name = 1;
257   }
258   if (is_watched_name) {
259     r = bpf_map_lookup_elem(&watched_path, cached_path);
260     if (r) {
261       __builtin_memset(&r->current_ikey, 0, sizeof(r->current_ikey));
262     }
263   }
```

Listing 5.2: Scelta dello stato da aggiornare in `handle_path_unlink`

5.3.3 `path_rename`

In `path_rename`, così come in `path_link`, non sono coinvolte solo le informazioni di un file ma di due, quello di sorgente e quello di destinazione. Questi hook vengono infatti invocati dal kernel durante le system call rename e link, eseguite ad esempio dai comandi `mv` e `ln`.

Sul lato sorgente il problema è lo stesso dell'unlink ed è stato risolto allo stesso modo. Quando si sposta il nome sorvegliato viene azzerato solo il legame tra path e inode e l'oggetto resta in `resolved_inode`, perché il rename non cambia l'inode e l'oggetto continua a esistere con un altro nome. Quando invece è la destinazione a essere un path sorvegliato viene rimosso da `resolved_inode` l'oggetto associato al path, perché il rename lo sovrascrive e quindi lo elimina senza passare dall'unlink. Anche qui il numero di inode dell'oggetto sovrascritto viene liberato e potrebbe essere riciclato.

5.3.4 `path_link`

In `handle_path_link` non è stato necessario modificare lo stato delle mappe. Infatti nel caso tipico, ovvero l'aggiunta di un hard link, viene solo aggiunto un nome allo stesso oggetto. Come per gli altri hook, la creazione del nuovo nome viene comunque segnalata lato user space. È stato aggiunto anche il controllo sul lato destinazione,

così da segnalare il caso in cui un path sorvegliato, dopo essere stato rimosso, viene ripopolato con un hard link verso un file esterno. Anche in questo caso non è necessario modificare le mappe. Un link infatti non può sovrascrivere un nome esistente [15], quindi quando viene creato il path sorvegliato era già stato rimosso da un unlink o da un rename, e in entrambi i casi il legame con il vecchio oggetto è già stato sciolto dai rispettivi hook. Il nuovo oggetto viene poi associato al path alla prima apertura, attraverso il riallineamento descritto in precedenza.

5.4 La semantica scelta per gli hard link

Con questa impostazione è stato necessario precisare, rispetto alle semantiche descritte nel Capitolo 3, cosa si intende quando l'utente indica un path da sorvegliare. Considerando solo il nome, le aperture tramite un hard link non verrebbero segnalate. Si è scelto invece di considerare anche l'oggetto che corrisponde al path, per cui le aperture attraverso altri nomi vengono segnalate e l'oggetto continua a essere seguito anche dopo la rimozione del nome originale, finché esiste almeno un altro nome che lo raggiunge.

La scelta è coerente con l'uso che si fa del programma. Per l'utente è intuitivo indicare una risorsa tramite il suo path, ma quello che vuole proteggere è il contenuto a cui il path dà accesso, e da uno strumento di sicurezza e di monitoraggio ci si aspetta l'osservazione di tutti i punti di accesso alla risorsa.

Questo comportamento però fa sì che a una stessa regola possano essere legati più oggetti. Se il nome sorvegliato viene rimosso mentre esiste un hard link e poi viene ricreato, alla regola restano associati sia il nuovo file, raggiunto tramite il path sorvegliato, sia il vecchio oggetto, ancora raggiungibile tramite l'hard link. Lo stesso accade quando il file sorvegliato viene rinominato e il path viene ricreato, come nella Figura 5.1. Dal punto di vista della sicurezza è il comportamento corretto, dato che il vecchio oggetto contiene ancora i dati della risorsa originale, ma il numero di oggetti associati a una regola può crescere e una voce viene rimossa da `resolved_inode` solo quando l'oggetto corrispondente smette di esistere.

5.5 Test

Per valutare la nuova versione sono stati ripetuti gli stessi test del capitolo precedente seguendo la stessa procedura. A differenza della versione precedente ora anche le operazioni sul filesystem generano eventi, quindi per ogni caso vengono riportati i comandi eseguiti e la sequenza di eventi prodotta, con il valore del campo MATCH di ciascun evento. In tutti i casi il file sorvegliato è `prova.txt` e nelle tabelle sono omesse le aperture con `cat` eseguite prima dell'operazione.

Sono stati inoltre aggiunti alcuni test specifici per i nuovi hook, in modo da verificare i casi in cui un oggetto ha più nomi o in cui il path sorvegliato viene svuotato e poi ripopolato. Sono state provate la rimozione di un nome diverso da

quello sorvegliato, la rimozione del nome sorvegliato quando esiste un hard link, la ricreazione del file in queste condizioni, il ripopolamento del path con un hard link verso un file esterno e il rename sia del nome sorvegliato sia di un altro nome dell'oggetto. Con un ultimo test si è verificato che, dopo una modifica con vim, il file estraneo che eredita il vecchio numero di inode non venga più considerato una risorsa sorvegliata.

Per `file_open` il campo MATCH mantiene il significato descritto nella Tabella 5.1, mentre per gli altri hook indica quale lato dell'operazione coinvolge una risorsa sorvegliata. Per `path_rename` e `path_link` il valore `ino` indica che la sorgente è un oggetto presente in `resolved_inode`, il valore `path` che la destinazione è un path presente in `watched_path` e il valore `both` che lo sono entrambe. Per `path_unlink` il valore è sempre `ino`, perché l'hook viene considerato solo quando l'oggetto rimosso è presente in `resolved_inode`.

5.6 Risultati

La Tabella 5.2 riporta i risultati dei test del capitolo precedente ripetuti con la nuova versione, confrontati con quelli ottenuti in precedenza. Nella tabella sono riportati solo i casi in cui il comportamento cambia, mentre l'apertura semplice, i due casi con i link simbolici e la directory padre rinominata producono gli stessi eventi della versione precedente.

Tabella 5.2: Risultati della strategia ibrida sui test del capitolo precedente in cui il comportamento cambia

Caso	Comandi	Eventi	Rispetto a prima
Modifica con vim	`vim prova.txt` `cat prova.txt`	RENAME (`ino`) OPEN W (`path`, `replaced`) UNLINK (`ino`) OPEN (`both`)	La sostituzione viene riconosciuta e il vecchio inode esce da `resolved_inode` quando la copia viene cancellata.

Sostituzione con `mv`	`mv nuovo.txt prova.txt`
	`cat prova.txt`
	`cat prova.txt`
		RENAME (`path`)
		OPEN (`path`, `replaced`)
		OPEN (`both`)
			Prima la sostituzione non generava eventi.

Unlink e ricreazione	`rm prova.txt`
	`touch prova.txt`
	`cat prova.txt`
		UNLINK (`ino`)
		OPEN W (`path`, `replaced`)
		OPEN (`both`)
			La sostituzione viene riconosciuta anche se il numero di inode coincide.

Apertura tramite hard link	`ln prova.txt hard.txt` `cat hard.txt`	LINK (`ino`) OPEN (`ino`)	La creazione del link viene segnalata nel momento in cui avviene.

Nel caso di vim, sulla macchina di test l'originale viene spostato in una copia con lo stesso nome seguito da una tilde, viene creato un nuovo file e alla fine la copia viene cancellata. Il rename sul lato sorgente scioglie il legame tra il path e il vecchio inode senza rimuovere l'oggetto da `resolved_inode`, l'apertura in scrittura del nuovo file viene riconosciuta per path e marcata come `replaced` e infine l'unlink della copia rimuove il vecchio inode quando l'oggetto smette di esistere. In questo modo all'apertura successiva con `cat` percorso e oggetto risultano di nuovo allineati.

Allo stesso modo vengono ora riconosciute la sostituzione con `mv` e la ricreazione dopo un unlink. In quest'ultimo caso la ricreazione del file ha riutilizzato lo stesso numero di inode, ma l'evento viene comunque marcato come `replaced` perché il legame con il vecchio oggetto era già stato sciolto al momento della cancellazione, quindi il riconoscimento non dipende più dalla coincidenza del numero di inode.

I risultati dei test aggiunti per i nuovi hook sono invece riportati nella Tabella 5.3.

Tabella 5.3: Risultati dei test aggiunti per gli hook `path_unlink`, `path_rename` e `path_link`

Caso	Comandi	Eventi	Esito
Inode riciclato dopo vim	`vim prova.txt` creazione di nuovi file fino a riassegnare il vecchio inode `cat` del file che lo eredita	RENAME (`ino`) OPEN W (`path`, `replaced`) UNLINK (`ino`) nessun evento sul file estraneo	Nessun falso positivo.

Rimozione di un altro nome	`ln prova.txt hard.txt`
	`rm hard.txt`
	`cat prova.txt`
		LINK (`ino`)
		UNLINK (`ino`)
		OPEN (`both`)
			Le mappe restano intatte e l'apertura non viene marcata come `replaced`.

Rimozione del nome sorvegliato	`ln prova.txt hard.txt`
	`rm prova.txt`
	`cat hard.txt`
		LINK (`ino`)
		UNLINK (`ino`)
		OPEN (`ino`)
			L'oggetto resta sorvegliato tramite l'altro nome.

Ricreazione dopo la rimozione del nome sorvegliato	`ln prova.txt hard.txt`
	`rm prova.txt`
	`echo nuovo > prova.txt`
	`cat prova.txt`
	`cat hard.txt`
		LINK (`ino`)
		UNLINK (`ino`)
		OPEN W (`path`, `replaced`)
		OPEN (`both`)
		OPEN (`ino`)
			Due oggetti legati alla stessa regola.

Hard link verso il path sorvegliato	`rm prova.txt`
	`echo roba > esterno.txt`
	`ln esterno.txt prova.txt`
	`cat prova.txt`
		UNLINK (`ino`)
		LINK (`path`)
		OPEN (`path`, `replaced`)
			Il ripopolamento viene segnalato e `esterno.txt`, che aveva ereditato il vecchio numero di inode, non viene confuso con la risorsa.

Rename del nome sorvegliato	`mv prova.txt spostato.txt` `cat spostato.txt`	RENAME (`ino`) OPEN (`ino`)	L'oggetto viene seguito con il nuovo nome.

Rename di un altro nome	`ln prova.txt hard.txt`
	`mv hard.txt altrove.txt`
	`cat prova.txt`
		LINK (`ino`)
		RENAME (`ino`)
		OPEN (`both`)
			Viene riportato il nome effettivamente spostato e le mappe restano intatte.

Rename del nome sorvegliato e ricreazione	`mv prova.txt spostato.txt`
	`echo nuovo > prova.txt`
	`cat prova.txt`
	`cat spostato.txt`
		RENAME (`ino`)
		OPEN W (`path`, `replaced`)
		OPEN (`both`)
		OPEN (`ino`)
			Entrambi gli oggetti restano sorvegliati.

Dal test sul riciclo del numero di inode, che era il problema principale della versione precedente, risulta che il file estraneo che eredita il vecchio numero non

genera più nessun evento, perché la voce corrispondente viene rimossa da `resolved_inode` nel momento in cui l'oggetto smette di esistere. Lo stesso emerge dal test con l'hard link verso il path sorvegliato, in cui `esterno.txt` aveva ereditato proprio il numero di inode della risorsa cancellata ma viene riconosciuto solo tramite il path e marcato come sostituzione.

I test sugli hard link confermano invece la semantica descritta nella sezione precedente. La rimozione di un nome diverso da quello sorvegliato non modifica lo stato, mentre la rimozione del nome sorvegliato scioglie solo il legame con il path e l'oggetto continua a essere riconosciuto tramite l'altro nome. Quando il path viene poi ricreato alla regola restano legati sia il nuovo file sia il vecchio oggetto.

5.7 Considerazioni

Con la strategia ibrida vengono risolti i problemi emersi nel capitolo precedente. Le sostituzioni vengono riconosciute sia all'apertura sia nel momento in cui avvengono, il riciclo del numero di inode non produce più falsi positivi e un oggetto sorvegliato viene seguito anche quando cambia nome o viene raggiunto con nomi diversi.

Rimangono però alcuni casi scoperti. Il rename di una directory lungo il percorso non viene segnalato come un'operazione su una risorsa sorvegliata, perché gli hook confrontano solo il nome effettivamente spostato, che in questo caso è quello della directory, e anche se il file continua a essere riconosciuto per inode il path salvato in `watched_path` non esiste più. Non è stato intercettato nemmeno l'hook `path_symlink`, perché la creazione di un link simbolico non dà di per sé accesso alla risorsa e l'apertura successiva viene comunque riconosciuta, perché il kernel risolve il link prima di invocare `file_open`, ma la creazione non viene segnalata.

Tutti i test descritti sono stati eseguiti sull'host, mentre il comportamento del programma in contesti diversi, come i container, viene affrontato nel capitolo successivo.

Capitolo 6

Container

Nel Capitolo 3 si è visto che un path viene risolto a partire dalla vista del filesystem del processo, per cui lo stesso path può indicare oggetti diversi per processi diversi. Un contesto in cui questo accade è quello dei container, e per questo la versione del prototipo descritta nel Capitolo 5, che nei test eseguiti sull'host ha risposto nel modo atteso, è stata provata anche su una macchina che esegue dei container. L'applicazione di uno strumento di monitoraggio ha particolare utilità proprio in questo contesto, come ad esempio in un server che ospita diversi servizi, perché può aiutare a individuare ed eventualmente prevenire escalation di privilegi e accessi a risorse protette. La versione finale del prototipo che ne è derivata corrisponde al tag `v4-namespace` del repository.

6.1 Namespace e container

Un namespace racchiude una risorsa globale del sistema in un'astrazione che fa apparire ai processi al suo interno di avere una propria istanza isolata di quella risorsa, per cui le modifiche fatte all'interno del namespace sono visibili solo ai processi che ne fanno parte. Linux prevede diversi tipi di namespace, tra cui quelli per i mount, per gli identificativi dei processi, per la rete e per gli utenti, e ogni namespace di un processo è rappresentato da un collegamento simbolico in `/proc/<pid>/ns/`, per cui due processi appartengono allo stesso namespace se questi collegamenti hanno lo stesso device e lo stesso numero di inode [20].

Il mount namespace è quello più rilevante per questo lavoro, perché isola la lista dei mount visti dai processi che ne fanno parte, e processi in mount namespace diversi possono quindi vedere gerarchie di directory completamente diverse [18]. Insieme alla directory radice di ogni processo, è questo il meccanismo per cui, come anticipato nel Capitolo 3, lo stesso path può indicare oggetti diversi a seconda del processo che lo risolve.

Un container non è un oggetto specifico del kernel, ma un insieme di processi eseguiti in namespace dedicati, all'interno di un proprio cgroup, cioè un gruppo di processi su cui il kernel limita e misura l'uso delle risorse come la CPU e la

memoria [21], e con una directory radice che corrisponde al filesystem del container. Nel caso di Docker la creazione vera e propria del container, cioè dei namespace, dei mount e della nuova radice, è affidata a `runc`, che poi avvia al suo interno il processo richiesto. Con l'opzione `-v` è possibile rendere visibile nel container un file o una directory dell'host attraverso un bind mount, che mostra lo stesso oggetto, con lo stesso device e lo stesso inode, in un altro punto dell'albero dei file [22]. Un container avviato con l'opzione `--privileged` riceve invece tutte le capability e l'accesso ai device dell'host [23], e tra le capability è presente `CAP_SYS_ADMIN`, che permette tra le altre cose di eseguire `mount` [24].

Infine i processi all'interno dei container vengono avviati dal demone di Docker e non da una sessione di login, per cui nei loro eventi il loginuid vale 4294967295, cioè il valore che indica che l'identificativo di login non è stato impostato.

6.2 Primi test

Per i primi test il programma è stato eseguito sulla macchina host, mentre l'apertura dei file è avvenuta all'interno di container creati con Docker. Sono stati provati quattro casi, uno in cui il container non ha accesso ai file dell'host, due in cui gli viene reso visibile un singolo file tramite un bind mount, allo stesso path o a un path diverso, e uno in cui il container monta l'intero filesystem dell'host. Non è stata testata la configurazione contraria, cioè con il programma all'interno del container e l'apertura dei file sull'host, perché non si tratta del caso d'uso immaginato per uno strumento di monitoraggio.

Il programma nella versione del capitolo precedente ha mostrato diversi problemi già nel primo test. Se ad esempio il programma sorveglia `/etc/shadow` sull'host e all'interno del container viene aperto il file `/etc/shadow` del container stesso, l'evento viene comunque segnalato. Questo accade perché `bpf_path_d_path()` costruisce il path a partire dalla directory radice del processo che ha aperto il file, e per un processo all'interno del container la radice è il filesystem del container stesso, per cui la funzione restituisce lo stesso path sia dentro sia fuori dal container nonostante i due oggetti siano diversi. Dato che il path corrisponde mentre la coppia (device, inode) è diversa, il programma interpreta l'apertura come una sostituzione del file, la marca come `replaced` e aggiorna le mappe con l'inode del file del container, perdendo il legame con il file reale dell'host. Poiché le mappe contengono ora l'oggetto del container, quando il container viene spento il programma intercetta anche un `UNLINK` su quel file e lo riporta nel log come se fosse stato cancellato il file originale.

Nel secondo test all'interno del container è stato reso visibile solo il file sorvegliato, con l'opzione `-v /prova/sensibile.txt:/prova/sensibile.txt`, mentre il programma era attivo sull'host. In questo caso il problema compare se il programma è già in esecuzione quando il container viene creato, situazione che corrisponde a un caso d'uso reale. Durante la creazione del container `runc` apre infatti un file segnaposto all'interno del filesystem del container, necessario perché il bind mount di un singolo file abbia un punto di aggancio. Questo file ha lo stesso path ma un

inode diverso, per cui corrisponde per path e il riallineamento elimina da `resolved_inode` l'inode del file reale, e un'apertura successiva risulta quindi `path` invece di `both` e viene marcata come `replaced`, anche se il file non è mai cambiato. Se invece il container è già avviato quando viene lanciato il programma, il comportamento è quello atteso, cioè un solo evento `both` senza `replaced`, perché device e inode sono quelli della risorsa. Il file viene quindi riconosciuto anche se l'apertura avviene da un mount namespace diverso da quello dell'host, e per questo escludere del tutto le corrispondenze per path provenienti da altri mount namespace, che avrebbe risolto il problema del primo test, non sarebbe stata una soluzione corretta.

Aprendo lo stesso file da dentro e da fuori il container si ottengono due eventi identici su device, inode e path, che si distinguono solo per mount namespace, pid namespace e cgroup del processo, per cui le informazioni riportate permettono comunque di distinguere l'origine dell'accesso.

Il terzo test è simile al secondo, ma il file dell'host viene montato nel container in un path diverso da quello sorvegliato, con `-v /prova/sensibile.txt:/dati/copia.txt`. In questo caso il riconoscimento non è risultato problematico e l'apertura è stata riconosciuta come `ino`, riportando sia il path usato nel container sia la risorsa della policy a cui corrisponde. Il criterio per path fallisce perché il nome usato non compare nella configurazione, mentre l'identificazione per device e inode funziona, e non c'è nemmeno il problema legato a `runc` e al bind mount, perché il segnaposto viene creato nel path di destinazione, che non è sorvegliato.

L'ultimo test simula un'evasione dal container, uno scenario non improbabile quando un container viene eseguito con più privilegi del necessario. Un container avviato con `--privileged` riceve tutte le capability, fra cui `CAP_SYS_ADMIN`, e nel suo `/dev` compaiono i device dell'host, per cui può eseguire `mount` sul device che contiene il filesystem radice dell'host e rendere visibile nella propria vista tutto quello che non gli era concesso. È il caso in cui un servizio eseguito per errore all'interno di un container privilegiato riesce ad accedere alle risorse dell'host. Nel test il file viene aperto come `/host/prova/sensibile.txt`, un path diverso da quello definito nella policy, e l'apertura viene riconosciuta come `ino`, dato che device e inode sono identici a quelli della risorsa, per cui anche questo caso, particolarmente importante, viene individuato.

Nel complesso il criterio per device e inode funziona in tutti e quattro i casi, mentre è quello per path a produrre i problemi, perché il path dipende dal mount namespace in cui viene risolto, anche se rimane comunque necessario per riconoscere le sostituzioni del file. Dati i risultati, e quindi la necessità di modificare il codice, in questa prima fase non sono state testate configurazioni che coinvolgessero anche gli altri hook.

6.3 Revisione del prototipo

Per rendere il programma resistente anche in presenza di container è stata rivista un'ultima volta la struttura della strategia ibrida, rendendo la regola di aggiorna-

mento delle mappe più conservativa ma resistente a casi che comprendono mount namespace differenti.

La coppia (device, inode) rimane l'identità dell'oggetto ed è sufficiente a riconoscere un accesso alla risorsa indipendentemente dal path usato e dal mount namespace da cui avviene. Il path continua invece a rappresentare la risorsa indicata dall'utente nella policy, ma ha significato solo rispetto a una determinata vista del filesystem. Per questo è stato introdotto il mount namespace come informazione aggiuntiva, che non fa parte dell'identità dell'oggetto ma indica il contesto in cui il path è valido. Una corrispondenza per solo path può quindi aggiornare le mappe solo se avviene nello stesso mount namespace in cui è stata configurata la policy, mentre negli altri casi l'apertura viene segnalata senza modificare lo stato. Le regole che ne derivano per `file_open` sono riassunte nella Tabella 6.1.

Tabella 6.1: Regole di riconoscimento e di aggiornamento di `file_open` con il controllo sul mount namespace

Match	Mount namespace	Situazione	Azione del programma	Marcatore
`both`	qualsiasi	Il path e l'oggetto aperto corrispondono alla stessa risorsa.	L'apertura viene segnalata senza modificare le mappe.	nessuno
`path`	uguale a quello della policy	L'oggetto associato al path è cambiato nella stessa vista del filesystem in cui è stata configurata la policy.	Il nuovo inode sostituisce il precedente in entrambe le mappe.	`replaced`
`path`	diverso da quello della policy	Lo stesso path indica un oggetto diverso in un'altra vista del filesystem, ad esempio dentro un container.	L'apertura viene segnalata senza modificare le mappe.	`PATH_ COLLISION`
`ino`	qualsiasi	L'oggetto sorvegliato viene raggiunto con un altro nome, ad esempio tramite un hard link, un bind mount o il filesystem dell'host montato nel container.	L'apertura viene segnalata senza modificare le mappe.	`ALIAS_ ACCESS`
nessuno	qualsiasi	Il file non è sorvegliato.	Nessun evento.	–

6.4 Implementazione

Per implementare la logica il programma in user space legge il namespace in cui viene eseguito da `/proc/self/ns/mnt` all'avvio e lo salva, per ogni path, nel campo `mnt_ns` della regola associata in `watched_path`. Nel kernel invece si ricava il valore dal task corrente (`nsproxy->mnt_ns->ns.inum`) e dato che il numero di inode di `/proc/self/ns/mnt`, ottenuto con `stat()`, corrisponde proprio a questo identificativo, i due valori possono essere confrontati direttamente.

Il Listing 6.1 riporta la parte di `handle_file_open` in cui il riallineamento viene eseguito solo se l'apertura avviene nel mount namespace della policy.

```
task = bpf_get_current_task_btf();
BPF_CORE_READ_INTO(&cur_mnt_ns, task, nsproxy, mnt_ns, ns.inum);
same_ns = (cur_mnt_ns == r->mnt_ns);

if (by_path && !by_inode && same_ns) {
  inode_changed = 1;
  bpf_map_delete_elem(&resolved_inode, &r->current_ikey);
  bpf_map_update_elem(&resolved_inode, &ikey, &mem->clean_key, BPF_ANY);
  r->current_ikey = ikey;
}
```

Listing 6.1: Controllo sul mount namespace prima del riallineamento in `handle_file_open`

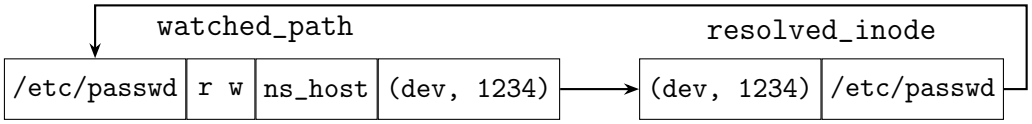
A ogni evento è stato inoltre aggiunto il campo `foreign_ns`, che indica se l'operazione è avvenuta in un mount namespace diverso da quello della policy, e in questo caso il programma in user space riporta nel log anche il namespace del processo. Il contenuto delle mappe con questa modifica è mostrato nella Figura 6.1.

Il controllo sul mount namespace è stato aggiunto anche negli hook che mantengono aggiornate le mappe.

In `path_unlink`, se il nome rimosso era l'ultimo link dell'oggetto, il legame tra path e inode viene sciolto e l'oggetto viene eliminato da `resolved_inode` indipendentemente dal namespace in cui avviene l'operazione, perché l'oggetto viene effettivamente distrutto e il suo numero di inode potrà essere riutilizzato. Se invece l'oggetto ha altri nomi e l'operazione avviene in un mount namespace diverso da quello della policy il legame non viene modificato, dato che il nome rimosso appartiene a un'altra vista del filesystem e non può essere confrontato con il path sorvegliato. Solo nel namespace della policy il nome rimosso viene confrontato con il path sorvegliato e il legame viene sciolto se i due coincidono.

In `path_rename` lo stesso controllo viene applicato sia al file spostato sia alla destinazione. Il legame del path sorvegliato con il suo inode viene sciolto quando il nome spostato è proprio il path sorvegliato, ma solo se il rename avviene nel namespace della policy. Allo stesso modo la destinazione viene considerata un path sorvegliato solo se il rename avviene nel namespace della policy, altrimenti un `mv` verso `/etc/shadow` all'interno di un container scioglierebbe il legame della policy dell'host. Se infine la destinazione è già un oggetto sorvegliato, che viene quindi

(a) Configurazione della policy dal mount namespace dell'host



(b) Apertura di /etc/passwd all'interno di un container

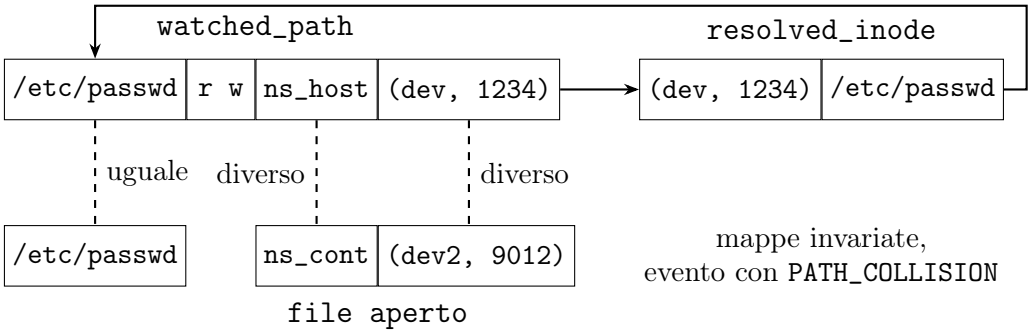


Figura 6.1: Contenuto delle due mappe dopo la configurazione della policy e all'apertura dello stesso path all'interno di un container

sovrascritto dal rename, vale la stessa regola di `path_unlink`. Per questo motivo nella versione finale il valore `ino` del campo MATCH di un rename indica che è sorvegliata la sorgente oppure l'oggetto che il rename sovrascrive nella destinazione, estendendo il significato descritto nel Capitolo 5.

`path_link` non modifica le mappe e usa il namespace solo per stabilire se il nuovo nome corrisponde a un path sorvegliato, corrispondenza che anche in questo caso viene considerata solo nel namespace della policy. In tutti gli hook, infine, l'evento inviato in user space ha il campo `foreign_ns` impostato quando l'operazione avviene in un mount namespace diverso da quello della policy.

6.5 Risultati

Con la nuova versione sono stati ripetuti i test sui container, aggiungendo dei casi in cui il file sorvegliato viene rinominato, cancellato o collegato con un hard link dall'interno di un container privilegiato che ha montato il disco dell'host in `/host`. In tutti i test il file sorvegliato è `/opt/segreto.txt` dell'host. La Tabella 6.2 riporta i casi in cui un container apre un file con lo stesso path della risorsa o la risorsa stessa, mentre la Tabella 6.3 riporta le operazioni eseguite sul file dell'host dall'interno del container privilegiato. Per ogni caso sono indicati, nell'ordine in cui sono stati eseguiti, i comandi, il lato da cui sono stati lanciati e gli eventi che ciascuno ha prodotto. Tutti gli eventi generati all'interno del container riportano anche il mount namespace del processo, che nelle tabelle è omesso.

Tabella 6.2: Aperture da container di un file con lo stesso path della risorsa o della risorsa stessa

Caso	Lato	Comando	Eventi
Il container crea e legge un proprio file	container	`echo interno > /opt/segreto.txt`	OPEN W (`path`, `PATH_COLLISION`) ×2
	container	`cat /opt/segreto.txt`	OPEN (`path`, `PATH_COLLISION`) ×2
	host	`cat /opt/segreto.txt`	OPEN (`both`)
	Le mappe non vengono modificate e l'host resta legato al file reale.		
Bind mount allo stesso path	host	`docker run -v /opt/segreto.txt: /opt/segreto.txt:ro`	OPEN di `runc` (`path`, `PATH_COLLISION`)
	container	`cat /opt/segreto.txt`	OPEN (`both`)
	Il segnaposto di `runc` non modifica più le mappe.		
Container già attivo	host	container con lo stesso bind mount avviato prima del programma	nessun evento
	container	`cat /opt/segreto.txt` con `docker exec`	OPEN (`both`)
	Invariato.		
Container privilegiato	container	`cat /host/opt/segreto.txt`	OPEN (`ino`, `ALIAS_ACCESS`)
	L'accesso al file dell'host viene riconosciuto.		

Tabella 6.3: Operazioni sul file dell'host eseguite dall'interno di un container privilegiato che ha montato il disco dell'host in `/host`

Caso	Lato	Comando	Eventi
Sostituzione dal container	container	`echo falso > /host/opt/nuovo.txt`	nessun evento
	container	`mv /host/opt/nuovo.txt /host/opt/segreto.txt`	RENAME (`ino`)
	host	`cat /opt/segreto.txt`	OPEN (`path`, `replaced`)
	host	`cat /opt/segreto.txt`	OPEN (`both`)
	La sostituzione viene segnalata e l'host si riallinea alla prima apertura.		
Rimozione dal container	container	`rm -f /host/opt/segreto.txt`	UNLINK (`ino`)
	host	`echo ricreato | tee /opt/segreto.txt`	OPEN W (`path`, `replaced`)
	host	`cat /opt/segreto.txt`	OPEN (`both`)
	Il nuovo file riceve lo stesso numero di inode ma viene trattato come un nuovo oggetto.		
Hard link dal container	container	`ln /host/opt/segreto.txt /host/opt/copia.txt`	LINK (`ino`)
	container	`cat /host/opt/copia.txt`	OPEN (`ino`, `ALIAS_ACCESS`)
	host	`cat /opt/copia.txt`	OPEN (`ino`, `ALIAS_ACCESS`)
	Il nuovo nome viene riconosciuto sia dal container sia dall'host.		
Rename nella vista del container	container	`echo x > /opt/altro.txt` `mv /opt/altro.txt /opt/segreto.txt`	nessun evento
	container	`cat /opt/segreto.txt`	OPEN (`path`, `PATH_COLLISION`) $\times 2$
	host	`cat /opt/segreto.txt`	OPEN (`both`)
	Il rename non modifica il legame dell'host.		
Accesso dal container dopo la sostituzione	container	`echo falso > /host/opt/nuovo.txt` `mv /host/opt/nuovo.txt /host/opt/segreto.txt`	RENAME (`ino`)
	container	`cat /host/opt/segreto.txt`	nessun evento
	host	`cat /opt/segreto.txt`	OPEN (`path`, `replaced`)
	container	`cat /host/opt/segreto.txt`	OPEN (`ino`, `ALIAS_ACCESS`)
	Il nuovo file non viene riconosciuto dal container finché l'host non lo riapre.		

I problemi osservati nei primi test non si presentano più. Il file del container con lo stesso path della risorsa produce solo eventi marcati come `PATH_COLLISION`, sia in scrittura sia in lettura, e l'apertura successiva dall'host risulta ancora `both`. Lo stesso vale per il segnaposto creato da `runc`, che viene aperto quando il processo si trova già nel mount namespace del container. Anche il rename eseguito nella vista del container verso `/opt/segreto.txt` non viene più considerato un'operazione sul path sorvegliato, per cui non genera eventi e non scioglie il legame dell'host.

Ogni apertura di un file del container produce però due eventi, che differiscono solo per il device. Il filesystem dei container Docker è infatti un overlayfs [25], che mostra al container la sovrapposizione di più directory, dette livelli, memorizzate sul disco dell'host. Quando un file dell'overlay viene aperto, overlayfs apre a sua volta il file reale nel livello in cui si trova, e anche questa seconda apertura passa dall'hook `file_open`. Il primo evento riporta quindi il device dell'overlay, assegnato dal kernel, e il secondo quello del disco dell'host, mentre il numero di inode è lo stesso, perché con tutti i livelli sullo stesso filesystem overlayfs riporta quello del file reale. Gli eventi doppi compaiono così solo quando il container apre un file del proprio filesystem, e non quando il file viene aperto dall'host o raggiunto attraverso un bind mount o il disco dell'host montato in `/host`. Non si tratta comunque di un problema, dato che entrambi gli eventi vengono classificati correttamente come `PATH_COLLISION` e nessuno dei due modifica le mappe, e l'unico effetto è quindi una segnalazione in più.

Gli accessi alla risorsa dell'host tramite il disco montato vengono invece riconosciuti come `ALIAS_ACCESS`, sia nel caso della semplice apertura sia attraverso un hard link creato dall'interno del container. Le operazioni che modificano davvero il file dell'host vengono segnalate anche quando partono da un altro namespace. Nella sostituzione con `mv` il vecchio oggetto viene sovrascritto e quindi distrutto, e nella cancellazione viene rimosso l'ultimo nome, per cui in entrambi i casi il legame viene sciolto e l'host si riallinea alla prima apertura del path.

Infine sono stati rieseguiti con la versione finale anche i test sull'host dei capitoli precedenti, per verificare che il controllo sul mount namespace non avesse introdotto problematiche. Gli eventi coincidono con quelli riportati nel Capitolo 5, con l'aggiunta del marcatore `ALIAS_ACCESS` sulle aperture riconosciute solo per inode. L'unica differenza è nel rename della sostituzione con `mv`, che ora risulta `both`, perché oltre al path di destinazione viene riconosciuto anche l'oggetto sorvegliato che il rename sovrascrive.

6.6 Considerazioni

Con il controllo sul mount namespace il prototipo riconosce correttamente i casi provati con i container. La coppia (device, inode) è risultata sufficiente a riconoscere la risorsa da qualunque vista del filesystem venga raggiunta, mentre il path ha significato solo nella vista in cui è stato definito.

L'unico limite emerso, mostrato nella Tabella 6.3, riguarda la sostituzione del file dall'interno di un container, dopo la quale il nuovo oggetto viene legato al path solo alla prima apertura dall'host e fino a quel momento non viene riconosciuto se raggiunto dal container. Il legame non può essere creato già al momento del rename, perché gli hook LSM vengono invocati prima che l'operazione sia completata e non è quindi garantito che questa vada a buon fine.

Le altre informazioni sul contesto del processo, come pid namespace e cgroup, vengono infine solo riportate negli eventi e non sono usate per prendere decisioni, un possibile sviluppo discusso nel Capitolo 7.

Capitolo 7

Conclusioni

In questo capitolo vengono riassunti i limiti del prototipo e i possibili sviluppi futuri, per poi trarre le conclusioni del lavoro svolto.

7.1 Limiti noti

Un primo limite è nella lunghezza massima che path e nomi dei file da sorvegliare possono avere, infatti i path vengono memorizzati in buffer di 256 byte, molto meno dei 4096 previsti da `PATH_MAX`, e i nomi sono limitati a 63 caratteri. Non si tratta di un'impostazione strutturale ma di una semplificazione data la natura del prototipo. Inoltre vengono intercettate solo l'apertura dei file e le operazioni di rinomina, cancellazione e creazione di link, per cui non vengono segnalate né le letture e le scritture su file già aperti prima dell'avvio del programma né altre operazioni, come la modifica dei permessi o del proprietario, e il rename di una directory lungo il percorso di un file sorvegliato non viene segnalato come un'operazione sulla risorsa.

A questi si aggiunge il limite nella sostituzione eseguita all'interno di un container descritto nel Capitolo 6.

La configurazione viene letta una sola volta all'avvio e richiede che i file sorvegliati esistano già in quel momento. Non sono state inoltre misurate le prestazioni del prototipo.

7.2 Possibili sviluppi futuri

Il prototipo si limita a segnalare gli accessi, ma gli hook LSM permettono anche di negare un'operazione, per cui un possibile sviluppo potrebbe essere quello di creare delle regole articolate e di bloccare a livello kernel le operazioni che le infrangono, oltre a segnalarle. Questo richiederebbe un sistema di riconoscimento affidabile e anche un sistema di feedback, in modo che l'utente possa intervenire tempestivamente in caso di blocchi non corretti.

Un altro aspetto, legato al primo, potrebbe essere quello di espandere l'insieme delle regole e delle politiche, soprattutto per far sì che diversi container possano

avere regole differenti, sfruttando il mount namespace, il pid namespace e il cgroup già riportati negli eventi. Infine il limite sulla lunghezza dei path potrebbe essere superato cambiando la chiave delle mappe, il prototipo potrebbe essere esteso ad altri hook e la configurazione potrebbe essere resa modificabile durante l'esecuzione.

7.3 Considerazioni finali

Il lavoro è partito dall'idea di realizzare un monitor degli accessi a file sensibili basato su eBPF, si è però concentrato sul modo in cui identificare in modo affidabile il file che l'utente vuole proteggere. Il percorso seguito per rispondere è stato sperimentale e ogni versione del prototipo è nata dai casi che la precedente non riusciva a gestire.

Il confronto iniziale ha mostrato che il path e l'inode non sono due modi equivalenti di indicare lo stesso file, ma identificano concetti diversi, cioè la risorsa che l'utente vuole proteggere e l'oggetto che in un dato momento le corrisponde, e che nessuno dei due da solo è sufficiente. La strategia ibrida ha permesso di tenerli insieme, mantenendo aggiornato il legame tra i due attraverso gli hook che intercettano le operazioni sul filesystem, e i test con i container hanno aggiunto un ultimo elemento, cioè che un path ha significato solo all'interno della vista del filesystem in cui viene risolto.

La versione finale riconosce correttamente i casi studiati e anche se non si tratta di uno strumento pronto all'uso, il modello a cui si è arrivati e i casi raccolti durante i test mostrano quali aspetti deve considerare uno strumento di questo tipo per non perdere traccia delle risorse che deve proteggere.

Appendice A

Uso del prototipo

A.1 Codice sorgente

Il codice del prototipo è disponibile nel repository [4]. Le versioni descritte nei capitoli precedenti sono contrassegnate dai tag riportati nella Tabella A.1, e possono essere recuperate con `git checkout` seguito dal nome del tag.

Tabella A.1: Tag del repository corrispondenti alle versioni del prototipo

Tag	Versione	Capitolo
`v1-inode`	Riconoscimento per (device, inode)	Capitolo 4
`v2-inode-path`	Criteri per (device, inode) e per path affiancati	Capitolo 4
`v3-hybrid`	Strategia ibrida con gli hook `path_unlink`, `path_rename` e `path_link`	Capitolo 5
`v4-namespace`	Versione finale con il controllo sul mount namespace	Capitolo 6

Il programma eBPF si trova nel file `src/fileopen.bpf.c`, il programma in user space in `src/fileopen.c` e le strutture condivise tra i due in `src/fileopen.h`. La libreria libbpf è inclusa come submodule git nella directory `deps/libbpf`.

A.2 Requisiti e compilazione

Per compilare ed eseguire il prototipo sono necessari un kernel che esporti il proprio BTF in `/sys/kernel/btf/vmlinux`, `bpf` tra i moduli di sicurezza attivi, l'opzione `CONFIG_SECURITY_PATH`, richiesta dagli hook `path_*`, e i pacchetti clang, make, bpftool, libelf e zlib. Il programma deve essere eseguito con i privilegi di root.

Dato che libbpf è inclusa come submodule, il repository va clonato insieme ai submodule. Prima della compilazione è necessario generare l'header `vmlinux.h` per il kernel in uso, dopodiché il prototipo si compila con `make`.

```
git clone --recurse-submodules https://github.com/lucamaffioli/ebpf-thesis.git
```

```
cd ebpf-thesis
bpftool btf dump file /sys/kernel/btf/vmlinux format c > vmlinux.h
make
```

A.3 Ambiente di sviluppo

Il prototipo è stato sviluppato e testato in una macchina virtuale x86_64 creata con Lima 2.2.0 su un host Fedora 44, con Ubuntu 26.04 LTS e kernel `7.0.0-31-generic`. Il kernel è compilato con le opzioni `CONFIG_BPF_LSM` e `CONFIG_DEBUG_INFO_BTF`, ma `bpf` non compare nella lista predefinita dei moduli di sicurezza impostata da `CONFIG_LSM`, per cui è stato aggiunto con il parametro di avvio `lsm=`, e la sua presenza tra i moduli attivi è stata verificata leggendo il file `/sys/kernel/security/lsm`. Il programma eBPF è stato compilato con clang 21.1.8, lo skeleton è stato generato con bpftool 7.7.0 e la versione di libbpf inclusa come submodule è la 1.7.0.

A.4 Utilizzo

Il programma riceve come argomenti i file da sorvegliare, ciascuno preceduto eventualmente da un'opzione che stabilisce quali accessi segnalare, cioè `-r` per le sole letture, `-w` per le sole scritture e `-rw` per entrambe, che è il comportamento predefinito. Ogni opzione vale per i file che la seguono. Ad esempio il primo dei comandi seguenti sorveglia tutti gli accessi a `/etc/shadow`, mentre il secondo segnala le letture di `/etc/shadow` e le scritture di `/etc/sudoers`.

```
sudo ./fileopen /etc/shadow
sudo ./fileopen -r /etc/shadow -w /etc/sudoers
```

All'avvio il programma stampa, per ogni file, la configurazione che ha inserito nelle mappe, e poi un evento per ogni operazione rilevata. Il primo esempio riguarda l'apertura in lettura di un file sorvegliato attraverso il suo path.

```
watching /root/tests/prova.txt (ino=1060133 dev=265289729 mntns=4026531832, r w)

[2026-09-23T17:17:01.197] OPEN   | both | R  | comm="cat         " path="/root/tests/prova
    .txt"
pid=10829 ppid=10809 uid=0 ruid=0 loginuid=1000 dev=265289729 ino=1060133 cgroup=7935
    mntns=4026531832 pidns=4026531836
```

Il secondo esempio mostra invece un evento generato all'interno di un container privilegiato che ha montato il disco dell'host in `/host`, mentre il programma sull'host sorveglia `/opt/segreto.txt`.

```
[2026-09-23T17:18:08.577] OPEN   | ino  | R  | comm="cat         " path="/host/opt/segreto
    .txt" [ALIAS_ACCESS] watched="/opt/segreto.txt" [ns=4026532320]
```

```
pid=11809 ppid=11783 uid=0 ruid=0 loginuid=4294967295 dev=265289729 ino=2961 cgroup=17302
    mntns=4026532320 pidns=4026532558
```

Ogni evento riporta l'operazione intercettata (`OPEN`, `UNLINK`, `RENAME` o `LINK`), il criterio con cui il file è stato riconosciuto, cioè `ino` se per (device, inode), `path` se per path e `both` se per entrambi, e, per le aperture, se l'accesso è in lettura o in scrittura. Seguono il nome del programma, il path usato e le informazioni sul processo. Infine l'evento può contenere uno o più dei seguenti marcatori.

- `[ALIAS_ACCESS]` indica che il file sorvegliato è stato raggiunto con un nome diverso da quello indicato nella configurazione.
- `[replaced]` indica che il path sorvegliato corrisponde ora a un oggetto diverso, che prende il posto del precedente.
- `[PATH_COLLISION]` indica che lo stesso path è stato usato in un altro mount namespace per un file diverso.
- `[ns=N]` indica che l'operazione proviene da un mount namespace diverso da quello in cui è stata definita la configurazione.

Nel secondo esempio il file viene quindi riconosciuto solo per (device, inode), perché è raggiunto con un path diverso da quello sorvegliato, il campo `watched` indica il path della configurazione a cui corrisponde e il marcatore `[ns=...]` indica che l'accesso proviene da un altro mount namespace.

Il programma si limita a osservare e non nega mai un'operazione. Inoltre, dato che gli accessi vengono rilevati al momento dell'apertura, le letture e le scritture eseguite su un file che era già aperto prima dell'avvio del programma non vengono segnalate.

Bibliografia

[1] eBPF docs. https://docs.ebpf.io.

[2] Falco. https://falco.org.

[3] Tetragon – eBPF-based security observability and runtime enforcement. https://tetragon.io/.

[4] Luca Maffioli. ebpf-thesis – codice. https://github.com/lucamaffioli/ebpf-thesis.

[5] bpf-helpers(7) – Linux manual page. https://man7.org/linux/man-pages/man7/bpf-helpers.7.html.

[6] Andrii Nakryiko. BPF ring buffer. https://nakryiko.com/posts/bpf-ringbuf/.

[7] The Linux Kernel documentation. Linux security module usage. https://docs.kernel.org/admin-guide/LSM/index.html.

[8] The Linux Kernel documentation. LSM BPF programs. https://docs.kernel.org/bpf/prog_lsm.html.

[9] Andrii Nakryiko. BPF CO-RE (Compile Once – Run Everywhere). https://nakryiko.com/posts/bpf-portability-and-co-re/.

[10] libbpf. https://github.com/libbpf/libbpf.

[11] libbpf-bootstrap. https://github.com/libbpf/libbpf-bootstrap.

[12] makedev(3) – Linux manual page. https://man7.org/linux/man-pages/man3/makedev.3.html.

[13] mount(2) – Linux manual page. https://man7.org/linux/man-pages/man2/mount.2.html.

[14] unlink(2) – Linux manual page. https://man7.org/linux/man-pages/man2/unlink.2.html.

[15] link(2) – Linux manual page. https://man7.org/linux/man-pages/man2/link.2.html.

[16] rename(2) – Linux manual page. https://man7.org/linux/man-pages/man2/rename.2.html.

[17] path_resolution(7) – Linux manual page. https://man7.org/linux/man-pages/man7/path_resolution.7.html.

[18] mount_namespaces(7) – Linux manual page. https://man7.org/linux/man-pages/man7/mount_namespaces.7.html.

[19] realpath(3) – Linux manual page. https://man7.org/linux/man-pages/man3/realpath.3.html.

[20] namespaces(7) – Linux manual page. https://man7.org/linux/man-pages/man7/namespaces.7.html.

[21] cgroups(7) – Linux manual page. https://man7.org/linux/man-pages/man7/cgroups.7.html.

[22] Docker docs – bind mounts. https://docs.docker.com/engine/storage/bind-mounts/.

[23] Docker docs – running containers. https://docs.docker.com/engine/containers/run/.

[24] capabilities(7) – Linux manual page. https://man7.org/linux/man-pages/man7/capabilities.7.html.

[25] The Linux Kernel documentation. Overlay filesystem. https://docs.kernel.org/filesystems/overlayfs.html.